

Object Oriented Programming

OOPs = Object oriented programming structure

OOPs is a programming model or paradigm which revolves around the concept of "OBJECTS".

An **object** refers to the instance of the class, which contains the instance of the members and behaviors defined in the class template. In the real world, an object is an actual entity to which a user interacts, whereas class is just the blueprint for that object. So the objects consume space and have some characteristic behavior.

A **class** can be understood as a template or a blueprint, which contains some values, known as member data or member, and some set of rules, known as behaviors or functions. So when an object is created, it automatically takes the data and functions that are defined in the class.

It deals with internal designing not external. So it is just to help the programmer to prevent data from mishandling.

Simula is considered the first object-oriented programming language. The programming paradigm where everything is represented as an object is known as a truly object-oriented programming language.

There are several benefits to using object-oriented programming (OOP) in software development:

Modularity: OOP allows you to break a large and complex problem down into smaller and more manageable pieces, or "objects." This makes it easier to write, test, and maintain your code.

Reusability: OOP allows you to reuse code by creating new objects that are similar to existing ones. This can save time and effort when developing software.

Extensibility: OOP allows you to easily add new features to an existing codebase by creating new objects or modifying existing ones.

Maintainability: OOP makes it easier to track and fix errors in your code, as objects are self-contained and can be isolated and tested more easily than procedural code.

Scalability: OOP allows you to build software that can grow and evolve over time, as new objects can be created or added to the existing codebase as needed.

Oops, are based on a bottom-up approach, unlike the structural programming paradigm, which uses a top-down approach.

What are some other programming paradigms other than OOPs?

Programming paradigms refers to the method of classification of programming languages based on their features. There are mainly two types of Programming Paradigms:

Imperative Programming Paradigm

Declarative Programming Paradigm

Now, these paradigms can be further classified based:

1. Imperative Programming Paradigm: Imperative programming focuses on HOW to execute program logic and defines control flow as statements that change a program state. This can be further classified as:

a) Procedural Programming Paradigm: Procedural programming specifies the steps a program must take to reach the desired state, usually read in order from top to bottom.

b) Object-Oriented Programming or OOP: Object-oriented programming (OOP) organizes programs as objects that contain some data and have some behavior.

c) Parallel Programming: Parallel programming paradigm breaks a task into subtasks and focuses on executing them simultaneously at the same time.

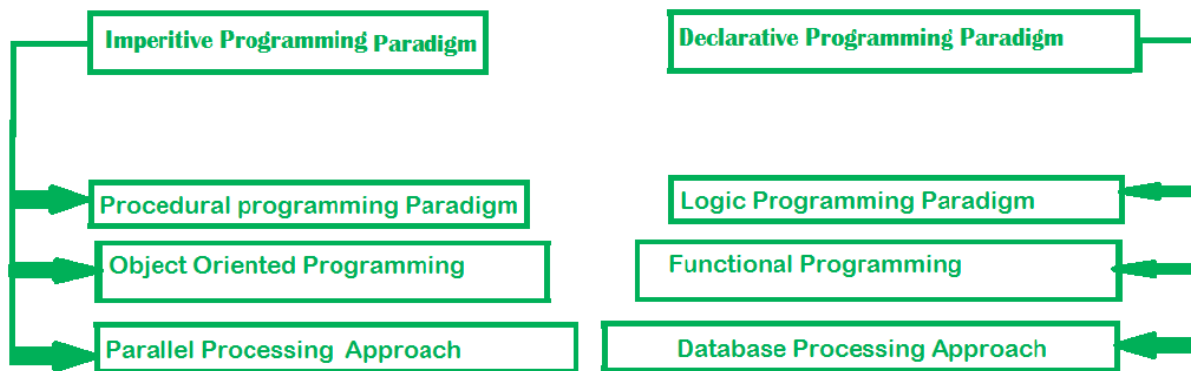
2. Declarative Programming Paradigm: Declarative programming focuses on WHAT to execute and defines program logic, but not a detailed control flow. Declarative paradigm can be further classified into:

a) Logical Programming Paradigm: Logical programming paradigm is based on formal logic, which refers to a set of sentences expressing facts and rules about how to solve a problem

b) Functional Programming Paradigm: Functional programming is a programming paradigm where programs are constructed by applying and composing functions.

c) Database Programming Paradigm: Database programming model is used to manage data and information structured as fields, records, and files. be created or added to the existing codebase as needed.

Programming Paradigms



Why do we need a Constructor in Oops?

Creating an object without a constructor is like buying a car with no color or we will color it after buying. Using Accessor and Mutators functions is philosophically wrong. Therefore the constructor must be there for initializing the object. At the time of placing the order I wanted a white color Scorpio or fortuner etc.

So the purpose of a constructor is to create a new object and set values for any existing object properties.

A constructor in C++ is a special method that is automatically called when an object of a class is created. To create a constructor, use the same name as the class, followed by parentheses ().

There should not be a return type of constructor.



```

1  //Accessor and Mutator
2  #include<bits/stdc++.h>
3  using namespace std;
4  class Marker{
5  private:
6      string color;
7      string nature;
8  public:
9      //Default Constructor or Non - parameterized Constructor
10     Marker(){
11         color = "Red";
12         nature = "Temporary";
13     }
14     //Parameterized Constructor
15     Marker(string color = "Red", string nature = "Temporary"){
16         this->color = color;
17         this->nature = nature;
18     }
19     //Copy Constructor
20     Marker(Marker &m){
21         this->color = m.color;
22         this->nature = m.nature;
23     }
24     pair<string ,string> getter(){
25         return {color,nature};
26     }
27 };
28 int main(){
29     Marker m1("Blue","Temporary"), m2("Black","Permanent");
30     //Marker m3;
31     //When we calling m3 without passing any thing then
32     //Error - call of overloaded 'Marker()' is ambiguous, so either comment default constructor
33     //Or Pass Atleast One Arguments
34     Marker m3(m2);
35     cout<<"Marker M1 - Color "<<m1.getter().first<<" Nature "<<m1.getter().second<<endl;
36     cout<<"Marker M2 - Color "<<m2.getter().first<<" Nature "<<m2.getter().second<<endl;
37     cout<<"Marker M3 - Color "<<m3.getter().first<<" Nature "<<m3.getter().second;
38     return 0;
39 }

```

```
≡ output.txt X
≡ output.txt
1  Marker M1 - Color Blue Nature Temporary
2  Marker M2 - Color Black Nature Permanent
3  Marker M3 - Color Black Nature Permanent
```

Deep Copy Constructor :-

```
1 //Deep Copy Constructor
2 #include<bits/stdc++.h>
3 using namespace std;
4 class Test{
5 private:
6     int a;
7     int *p;
8 public:
9     Test(int x){
10         a = x;
11         p = new int[5];
12     }
13     Test(Test &t){
14         this->a = t.a;
15         //this->p = t.p; // This is wrong.
16         this->p = new int[5]; //here we called deep copy constructor.
17     }
18 };
```

There are three types of constructors:

- Default / Non Parameterized Constructor - With no parameters. C++ says that a default constructor is automatically provided by the compiler.
- Parameterized Constructor - With Parameters, Create a new instance of a class and also pass arguments simultaneously.
- Copy Constructor - Which creates a new object as a copy of an existing object.

Type of Function inside Class :-

- Constructor
- Accessor
- Mutator
- Facilitators
- Enquiry/Inspector
- Destructor

The using Directive :-

A C++ program can be divided into different namespaces. A namespace is a part of the program in which certain names are recognized; outside of the namespace they're unknown.

The directive **using namespace std;**

says that all the program statements that follow are within the std namespace. Various program components such as cout are declared within this namespace.

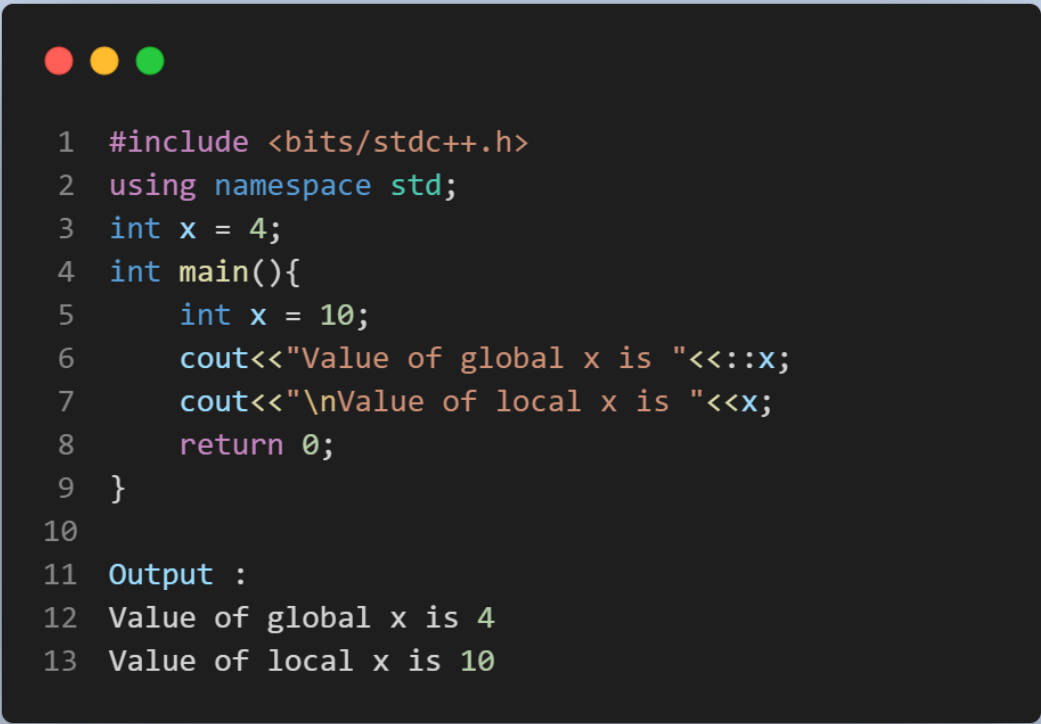
If we didn't use the using directive, we would need to add the std name to many program elements.

For example, in the FIRST program we'd need to say

```
std::cout << "Every age has a language of its own.";
```

Scope Resolution Operators (::) - it is used for the following purposes.

1. To access a global variable when there is a local variable with same name:



```
1  #include <bits/stdc++.h>
2  using namespace std;
3  int x = 4;
4  int main(){
5      int x = 10;
6      cout<<"Value of global x is "<<::x;
7      cout<<"\nValue of local x is "<<x;
8      return 0;
9  }
10
11 Output :
12 Value of global x is 4
13 Value of local x is 10
```

2.To Define a Function Outside the class.

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  class A{
4  private:
5      int a,b;
6  public:
7      A(int x, int y){
8          a=x,b=y;
9      }
10     int Multiplication();
11 };
12 int A::Multiplication(){
13     return a*b;
14 }
15 int main(){
16     A obj(5,6);
17     cout<<"The Multiplication is : "<<obj.Multiplication();
18     return 0;
19 }
20
21 Output :
22 The Multiplication is : 30
```

3.To access a class's static variables.

```

1  #include<bits/stdc++.h>
2  using namespace std;
3  class Test{
4      static int x;
5  public:
6      static int y;
7      void func(int x){
8          cout << "Value of static x is " << Test::x;
9          cout << "\nValue of local x is " << x;
10     }
11 };
12 // Static Member we can access directly without creating object.
13 int Test::x = 1;
14 int Test::y = 2;
15
16 int main(){
17     Test obj;
18     int x = 3 ;
19     obj.func(x);
20     cout << "\nTest::y = " << Test::y;
21     return 0;
22 }
23
24 Output:
25 Value of static x is 1
26 Value of local x is 3
27 Test::y = 2
28

```

4. In Case of multiple Inheritance: if the same variable name exists in two ancestor classes, we can use scope resolution operator to distinguish.



```
1  #include<bits/stdc++.h>
2  using namespace std;
3  class A{
4  protected:
5      int x;
6  public:
7      A() { x = 10; }
8  };
9  class B{
10 protected:
11     int x;
12 public:
13     B() { x = 20; }
14 };
15
16 class C: public A, public B{
17 public:
18     void fun(){
19         cout << "A's x is " << A::x;
20         cout << "\nB's x is " << B::x;
21     }
22 };
23 int main(){
24     C c;
25     c.fun();
26     return 0;
27 }
28
29 Output:
30 A's x is 10
31 B's x is 20
```

5. Refer to a class inside another class :- refer to a nested class using the scope resolution operator.

```
1  #include<bits/stdc++.h>
2  using namespace std;
3  class outside{
4  public:
5      int x;
6      class inside {
7      public:
8          int x;
9          static int y;
10         int foo();
11     };
12 };
13 int outside::inside::y = 5;
14 int main(){
15     outside A;
16     outside::inside B;
17 }
18
```

Inline Function :-

When a function is declared inline, the compiler places a copy of that specific function's code at each point where the function is called at compile time.

When the function is declared outside the class with inline references then this can become overhead if the execution time of function is less than the switching time from the caller function to called function (callee).

Note :- **Inlining is only a request to the compiler, not a command.**

Compiler can ignore the request for inlining.

Compiler may not perform inlining in such circumstances:

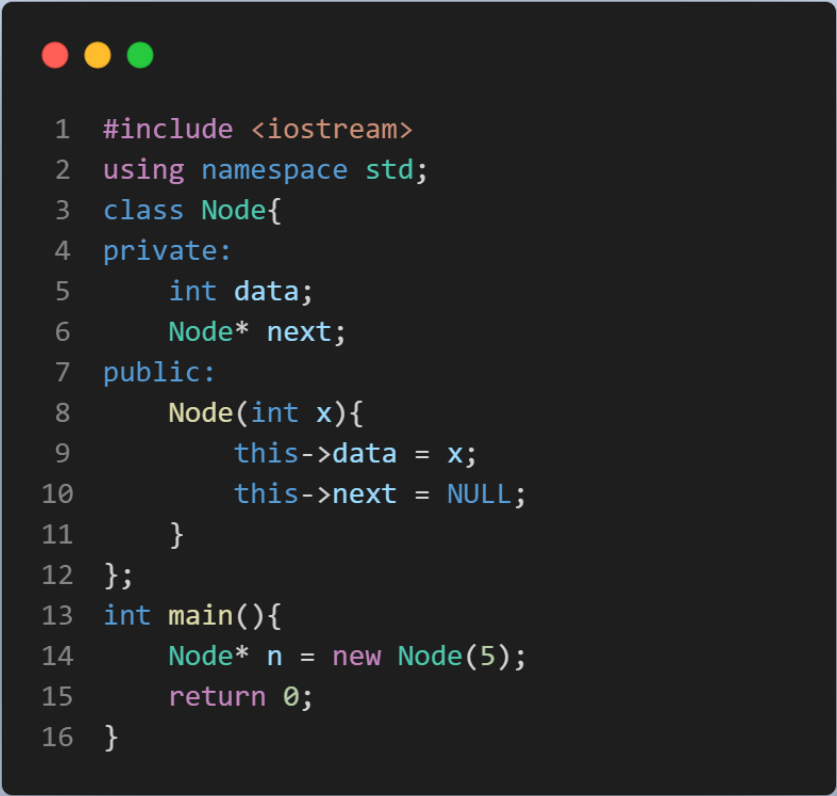
- If the function contains a loop.
- If a function contains static Variables
- If a function is recursive in nature.
- If a function return type is other than void
- If a function contains switch or goto statements.

```
1  #include <iostream>
2  using namespace std;
3  class operation
4  {
5      int a,b,add,sub,mul;
6      float div;
7  public:
8      void get();
9      void sum();
10     void difference();
11 };
12 inline void operation :: get(){
13     cout << "Enter first value:";
14     cin >> a;
15     cout << "Enter second value:";
16     cin >> b;
17 }
18 inline void operation :: sum(){
19     add = a+b;
20     cout << "Addition of two numbers: " << a+b << "\n";
21 }
22 inline void operation :: difference(){
23     sub = a-b;
24     cout << "Difference of two numbers: " << a-b << "\n";
25 }
26 int main(){
27     cout << "Program using inline function\n";
28     operation s;
29     s.get();
30     s.sum();
31     s.difference();
32     return 0;
33 }
34
```

This Pointer :- this is a keyword that refers to the current instance of the class.

There can be 3 main uses of "this" keyword :

- It can be used to declare indexers.
- It can be used to refer to the current class instance variable.
- It can be used to pass the current object as a parameter to another method.



```
1  #include <iostream>
2  using namespace std;
3  class Node{
4  private:
5      int data;
6      Node* next;
7  public:
8      Node(int x){
9          this->data = x;
10         this->next = NULL;
11     }
12 };
13 int main(){
14     Node* n = new Node(5);
15     return 0;
16 }
```

Structure	Class
-----------	-------

A Structure is defined with the struct keyword	A class is defined with the class keyword
All the member of a structure are public by default	All the members of a class are private by default
Structure can't be inherit	Class can be inherit
A structure contains only data member	A class contain both data member and member function
There is no data hiding features	Classes having data hiding features by using access specifiers(public, private, protected)

Operator Overloading :- we can make operators work for user-defined classes. This means C++ has the ability to provide the operators with a special meaning for a data type, this ability is known as operator overloading.

For example, we can overload an operator '+' in a class like String so that we can concatenate two strings by just using +. Other example classes where arithmetic operators may be overloaded are Complex Numbers, Fractional Numbers, Big Integer, etc.

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  class Complex{
4  private:
5      int real;
6      int img;
7  public:
8      Complex(int r = 0, int i = 0){
9          real = r;
10         img = i;
11     }
12     void display(){
13         cout << real <<" + i" << img << endl;
14     }
15     Complex add(Complex c){
16         Complex temp;
17         temp.real = real + c.real;
18         temp.img = img + c.img;
19         return temp;
20     }
21 };
22 int main(){
23     Complex c1(5, 3), c2(10, 5), c3;
24     c3 = c1.add(c2);
25     c3.display();
26 }
27
28 //Output
29 15 + i8
```

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  class Complex{
4  private:
5      int real;
6      int img;
7  public:
8      Complex(int r = 0, int i = 0){
9          real = r;
10         img = i;
11     }
12     void display(){
13         cout << real <<" + i" << img << endl;
14     }
15     Complex operator +(Complex c){
16         Complex temp;
17         temp.real = real + c.real;
18         temp.img = img + c.img;
19         return temp;
20     }
21     Complex operator -(Complex c){
22         Complex temp;
23         temp.real = real - c.real;
24         temp.img = img - c.img;
25         return temp;
26     }
27 };
28 int main(){
29     Complex c1(5, 3), c2(10, 5), c3, c4;
30     c3 = c2 + c1;
31     c4 = c2 - c1;
32     c4.display();
33     c3.display();
34 }
35
36 //Output
37 5 + i2
38 15 + i8
```

Operator Overloading using friend function :-

```
1 // Operator Overloading using Friend Functions
2 #include <iostream>
3 using namespace std;
4 class Complex{
5 private:
6     int real;
7     int img;
8 public:
9     Complex(int r = 0, int i = 0){
10         real = r;
11         img = i;
12     }
13     void display(){
14         cout << real <<" + i"<< img << endl;
15     }
16     friend Complex operator+(Complex c1, Complex c2);
17 };
18 Complex operator+(Complex c1, Complex c2){
19     Complex temp;
20     temp.real = c1.real + c2.real;
21     temp.img = c1.img + c2.img;
22     return temp;
23 }
24 int main(){
25     Complex c1(5, 3), c2(10, 6), c3;
26     c3 = c1 + c2;
27     // also written like this
28     // c3 = operator+(c1,c2);
29     c3.display();
30 }
31
32 // Output
33 15 + i9;
```

Ostream Operators Overloading :-

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  class Complex{
4  private:
5      int real;
6      int img;
7  public:
8      Complex(int r, int i){
9          real = r;
10         img = i;
11     }
12     friend ostream & operator<<(ostream &o, Complex &c);
13 };
14 ostream & operator<<(ostream &out, Complex &c){
15     out<<c.real<<" + i"<<c.img<<endl;
16     return out;
17 }
18 int main(){
19     Complex c1(4,6);
20     cout<<c1;
21     operator<<(cout,c1);
22     return 0;
23 }
24
25 // Output
26 4 + i6
27 4 + i6
```

Eg - Addition of Rational Numbers

```

1 // Program with a Class Rational Number
2 #include <iostream>
3 using namespace std;
4 class Rational{
5 private:
6     int p;
7     int q;
8 public:
9     Rational(){
10         p = 1;
11         q = 1;
12     }
13     Rational(int p, int q){
14         this->p = p;
15         this->q = q;
16     }
17     Rational(Rational &r){
18         this->p = r.p;
19         this->q = r.q;
20     }
21     int getP() { return p; }
22     int getQ() { return q; }
23     void setP(int p){
24         this->p = p;
25     }
26     void setQ(int q){
27         this->q = q;
28     }
29     Rational operator+(Rational r){
30         Rational t;
31         t.p = this->p * r.q + this->q * r.p;
32         t.q = this->q * r.q;
33         return t;
34     }
35     friend ostream &operator<<(ostream &os, Rational &r);
36 };
37 ostream &operator<<(ostream &os, Rational &r){
38     os << r.p << "/" << r.q;
39     return os;
40 }
41 int main(){
42     Rational r1(3, 4), r2(2, 5), r3;
43     r3 = r1 + r2;
44     cout << "Sum of " << r1 << " and " << r2 << " is " << r3 << endl;
45 }
46
47 // Output
48 Sum of 3/4 and 2/5 is 23/20

```

Inheritance

Inheritance :- It is one of the most important features of Object-Oriented Programming. The Capability of a class to derive properties and characteristics from another class is called inheritance.

Inheritance is a feature or a process in which, new classes are created from the existing classes. The new class created is called "Derived Class" or "Child Class" or "Sub Class" and the existing class is known as the "Base Class" or "Parent Class" or "Super Class".

When we say derived class inherits the base class, it means the derived class inherits all the properties of base class without changing the properties of base class and may add new features to its own derived class.

Why and when to use Inheritance.

- Inheritance is a term for reusing code by a mechanism of passing down information and behavior from a parent class to a child or subclass.
- Consider a group of vehicles. You need to create classes for Bus, Car, and Truck. The methods fuelAmount(), capacity(), applyBrakes(), will be the same for all three classes. If we create these classes avoiding inheritance then we have to write all of these functions in each of the three classes as shown below figure:-

Class Bus

fuelAmount()
capacity()
applyBrakes()

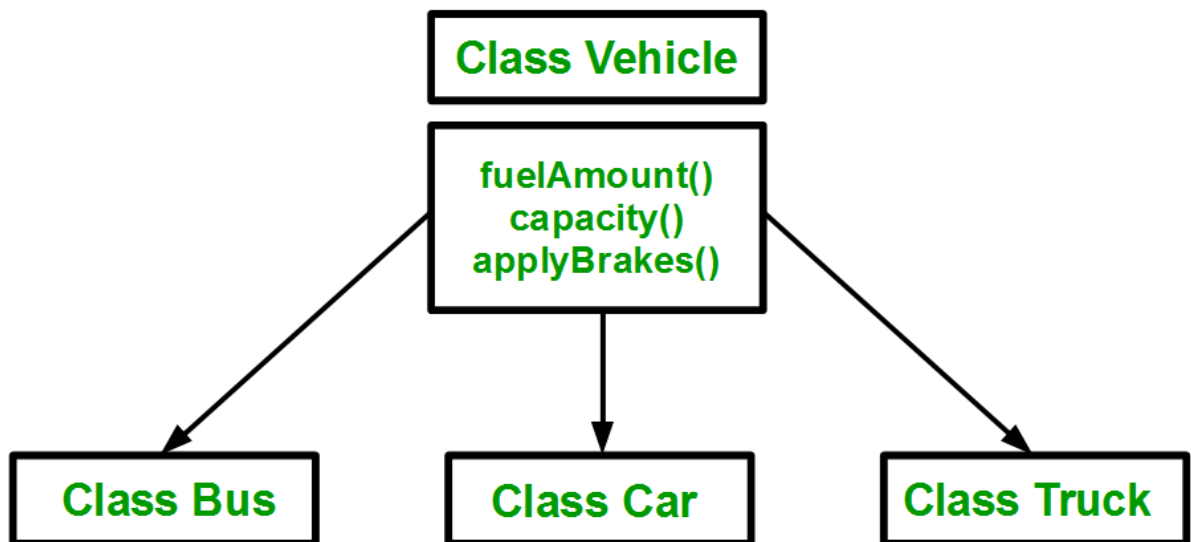
Class Car

fuelAmount()
capacity()
applyBrakes()

Class Truck

fuelAmount()
capacity()
applyBrakes()

- We can clearly see that the above process results in duplication of the same code 3 times. This increases the chances of error and data redundancy. To avoid this type of situation, inheritance is used.



- If you want the object to use all the behavior of the base class unless explicitly overridden, then inheritance is the simplest, least verbose, most straightforward way to express it.

- The main reason for using inheritance is not as a form of composition - it is so you can get polymorphic behavior. If you don't need polymorphism, you probably should not be using inheritance, at least in C++.

Available vs Accessible

Available : If a class Derived is inheriting from Base class then everything from Base class will be available/present in Derived class.

Accessible: everything is available in Derived class but private members are not accessible.

Example: bank balance of Father is available for Son by not accessible. Son has to ask Father for money. Only Father can access his account.

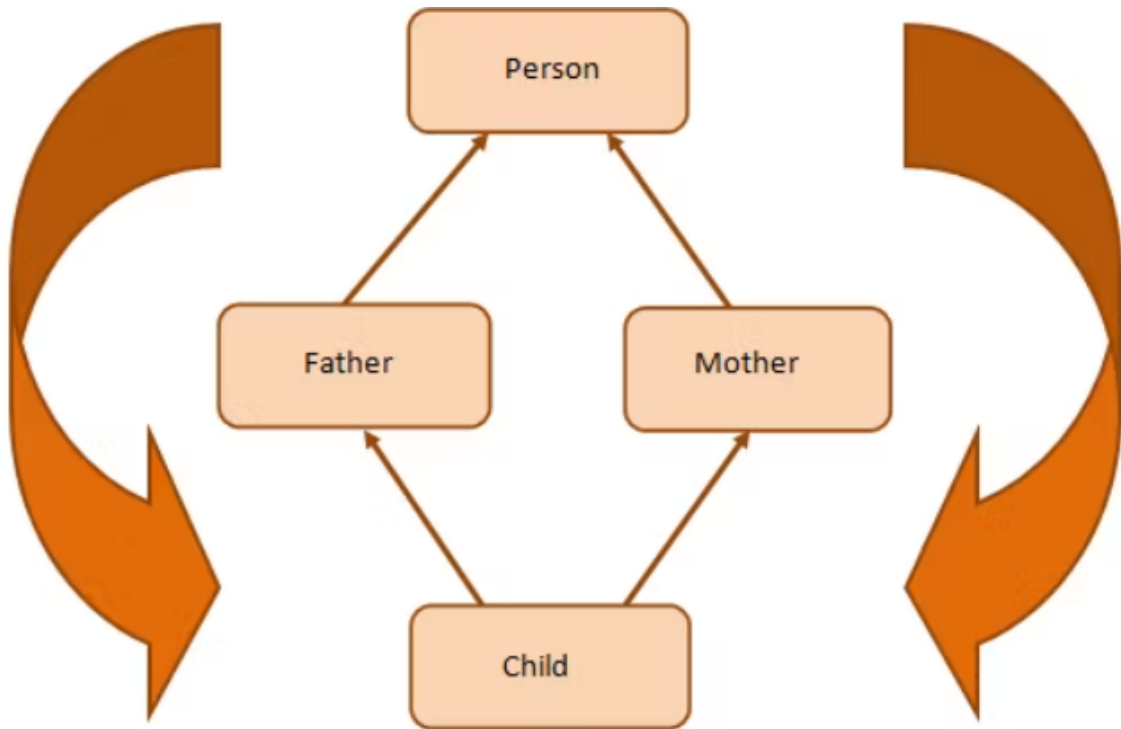
```
1 #include<bits/stdc++.h>
2 using namespace std;
3 // Example of inheritance
4 class Rectangle{
5 private:
6     int length;
7     int breadth;
8 public:
9     Rectangle(){length = 1,breadth=1;};
10    Rectangle(int l, int b){length = l,breadth=b;};
11    int getLength(){return length;};
12    int getBreadth(){return breadth;};
13    void setLength(int l){length = l;};
14    void setBreadth(int b){breadth = b;};
15    int area(){return length * breadth;};
16    int perimeter(){return 2 * (length + breadth);};
17    bool isSquare(){return length == breadth;};
18    ~Rectangle(){cout<<"Rectangle Destroyed";};
19 };
20 class Cuboid : public Rectangle{
21 private:
22     int height;
23 public:
24     Cuboid(int h){
25         height = h;
26     }
27     int getHeight() { return height; }
28     void setHeight(int h) { height = h; }
29     int volume() { return getLength() * getBreadth() * height; }
30 };
31 int main(){
32     Cuboid c(5);
33     c.setLength(10);
34     c.setBreadth(7);
35     cout << "Volume is " << c.volume() << endl;
36 }
37 //Output
38 Volume is 350
39 Rectangle Destroyed
40
```

Constructors in inheritance :-

- Constructors of base class are executed first then the constructor of derived class is executed.
- By default, a non-parameterised constructor of base class is executed.
- A parameterised constructor of base class must be called from derived class constructor.

```
1 // Example of calling parameterised constructor
2 #include <iostream>
3 using namespace std;
4 class Base{
5 public:
6     Base(){cout<<"Non-param Base"<<endl;}
7     Base(int x){cout<<"Param of Base "<<x<<endl;}
8 };
9 class Derived:public Base{
10 public:
11     Derived(){cout<<"Non-Param Derived"<<endl;}
12     Derived(int y){cout<<"Param of Derived "<<y<<endl;}
13     Derived(int x,int y):Base(x){
14         cout<<"Param of Derived "<<y<<endl;
15     }
16 };
17 int main(){
18     Derived d(5,10);
19     return 0;
20 }
21 //Output
22 Param of Base 5
23 Param of Derived 10
```

The Diamond Problem (Multiple-path inheritance) :- The Diamond problem occurs when a child class inherits from two parent classes who both share a common grandparent class.



As shown in the figure, class *Child* inherits the traits of class *Person* twice—once from *Father* and again from *Mother*. This gives rise to ambiguity since the compiler fails to understand which way to go.

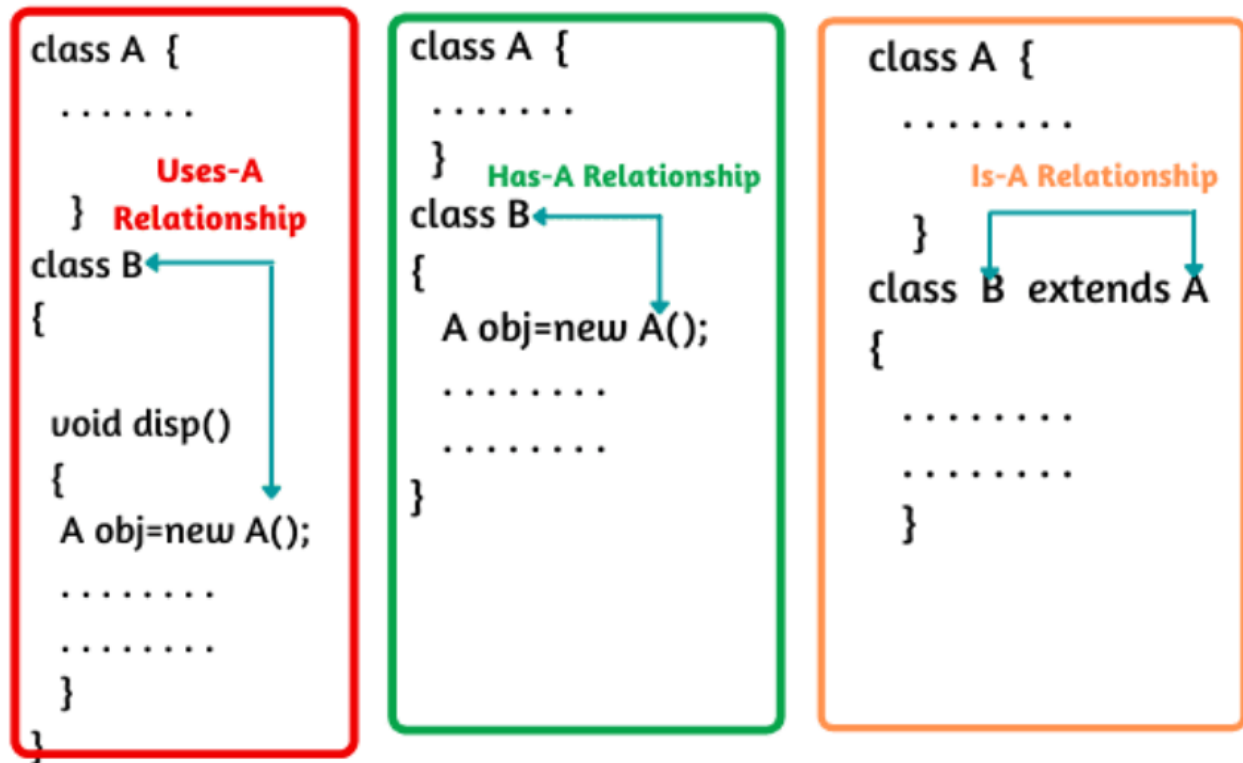


```
1 // Example of diamond problem
2 #include <iostream>
3 using namespace std;
4 class Person { //class Person
5 public:
6     Person(int x){cout << "Person::Person(int) called" << endl;}
7 };
8 class Father : public Person { //class Father inherits Person
9 public:
10     Father(int x):Person(x){
11         cout << "Father::Father(int) called" << endl;
12     }
13 };
14 class Mother : public Person{ //class Mother inherits Person
15 public:
16     Mother(int x):Person(x){
17         cout << "Mother::Mother(int) called" << endl;
18     }
19 };
20 class Child : public Father, public Mother{ //Child inherits Father and Mother
21 public:
22     Child(int x):Mother(x), Father(x) {
23         cout << "Child::Child(int) called" << endl;
24     }
25 };
26 int main(){
27     Child child(30);
28     return 0;
29 }
30
31 //Output
32 Person::Person(int) called
33 Father::Father(int) called
34 Person::Person(int) called
35 Mother::Mother(int) called
36 Child::Child(int) called
```

The solution to the diamond problem is to use the virtual keyword. We make the two parent classes (who inherit from the same grandparent class) into virtual classes in order to avoid two copies of the grandparent class in the child class

```
1 // Example of diamond problem
2 #include <iostream>
3 using namespace std;
4 class Person { //class Person
5 public:
6     Person(){cout << "Person::Person() called" << endl;}
7     Person(int x){cout << "Person::Person(int) called" << endl;}
8 };
9 class Father : virtual public Person { //class Father inherits Person
10 public:
11     Father(int x):Person(x){
12         cout << "Father::Father(int) called" << endl;
13     }
14 };
15 class Mother : virtual public Person{ //class Mother inherits Person
16 public:
17     Mother(int x):Person(x){
18         cout << "Mother::Mother(int) called" << endl;
19     }
20 };
21 class Child : public Father, public Mother{ //Child inherits Father and Mother
22 public:
23     Child(int x):Mother(x), Father(x){
24         cout << "Child::Child(int) called" << endl;
25     }
26 };
27 int main(){
28     Child child(30);
29     return 0;
30 }
31
32 //Output
33 Person::Person() called
34 Father::Father(int) called
35 Mother::Mother(int) called
36 Child::Child(int) called
```

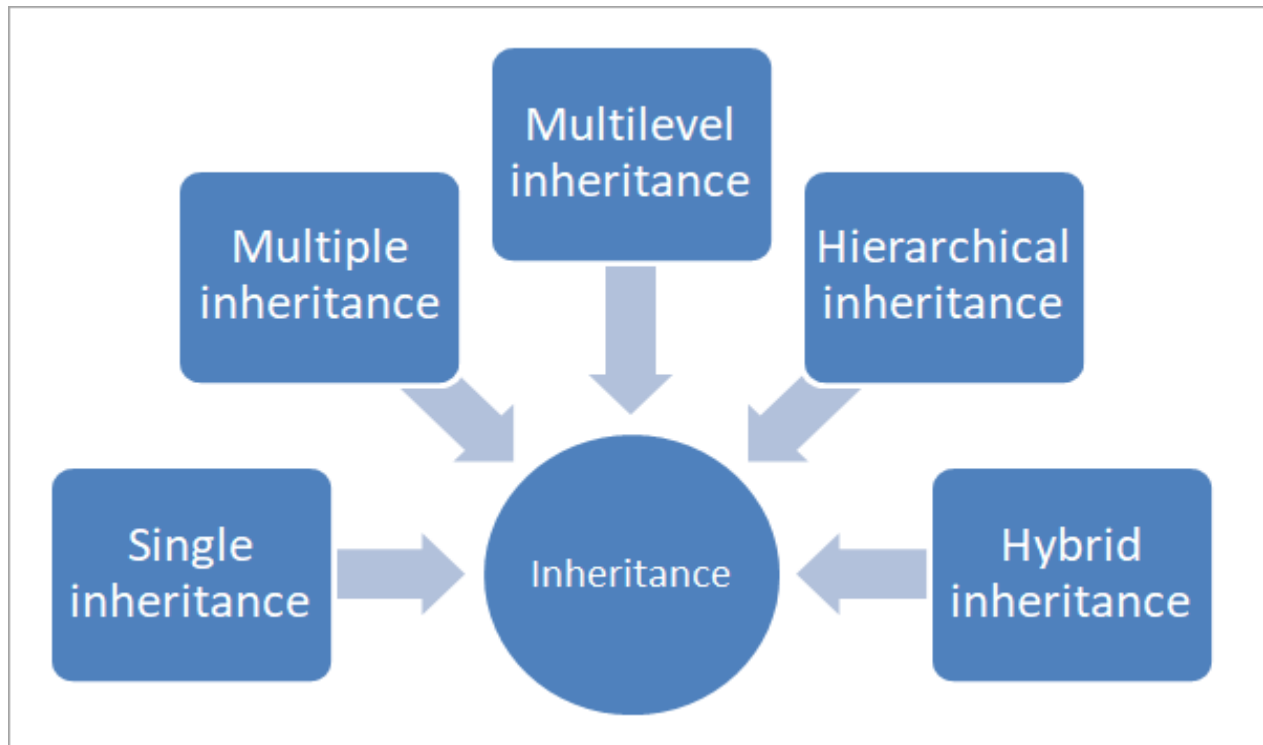
Uses-A, IsA and HasA Relationship :-



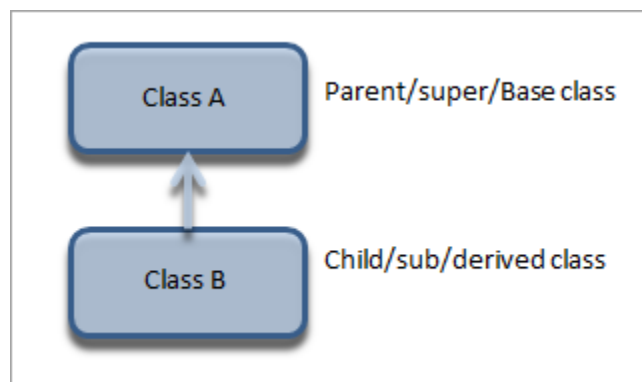
Access Specifiers in C++

Specifier	Within Same Class	In Derived Class	Outside the Class
Private	Yes	No	No
Protected	Yes	Yes	No
Public	Yes	Yes	Yes

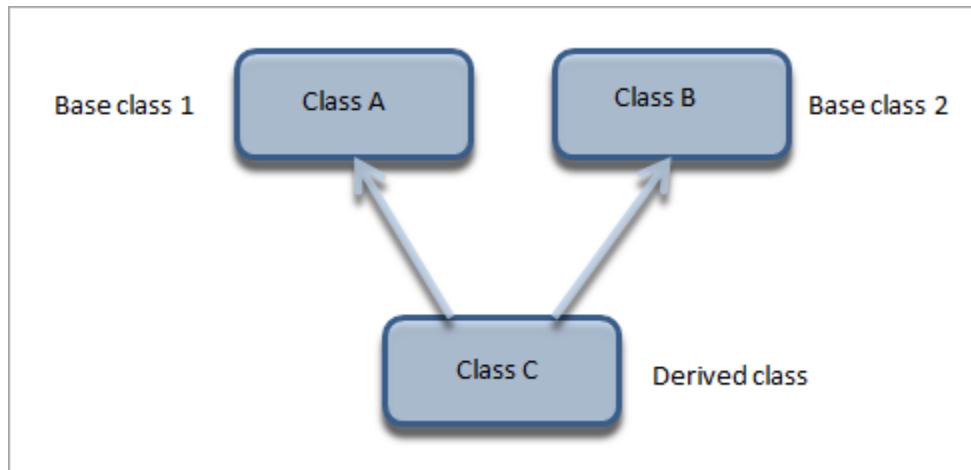
Types Of Inheritance :-



1) Single Inheritance :- In single inheritance, a class derives from one base class only. This means that there is only one subclass that is derived from one superclass.

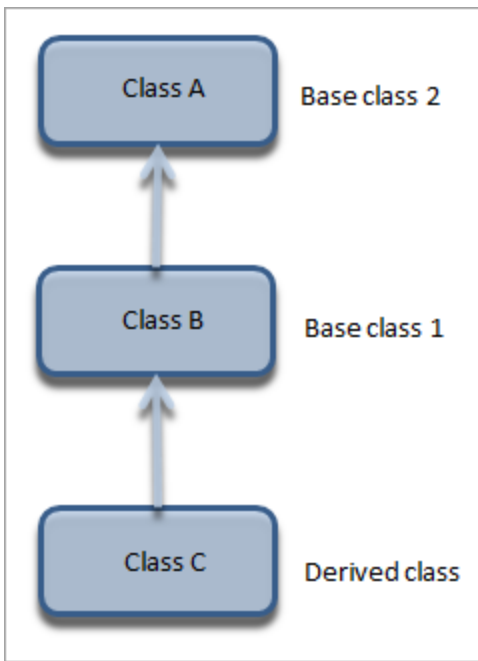


2) Multiple Inheritance :- Multiple inheritance is a type of inheritance in which a class derives from more than one class. As shown in the above diagram, class C is a subclass that has class A and class B as its parent.

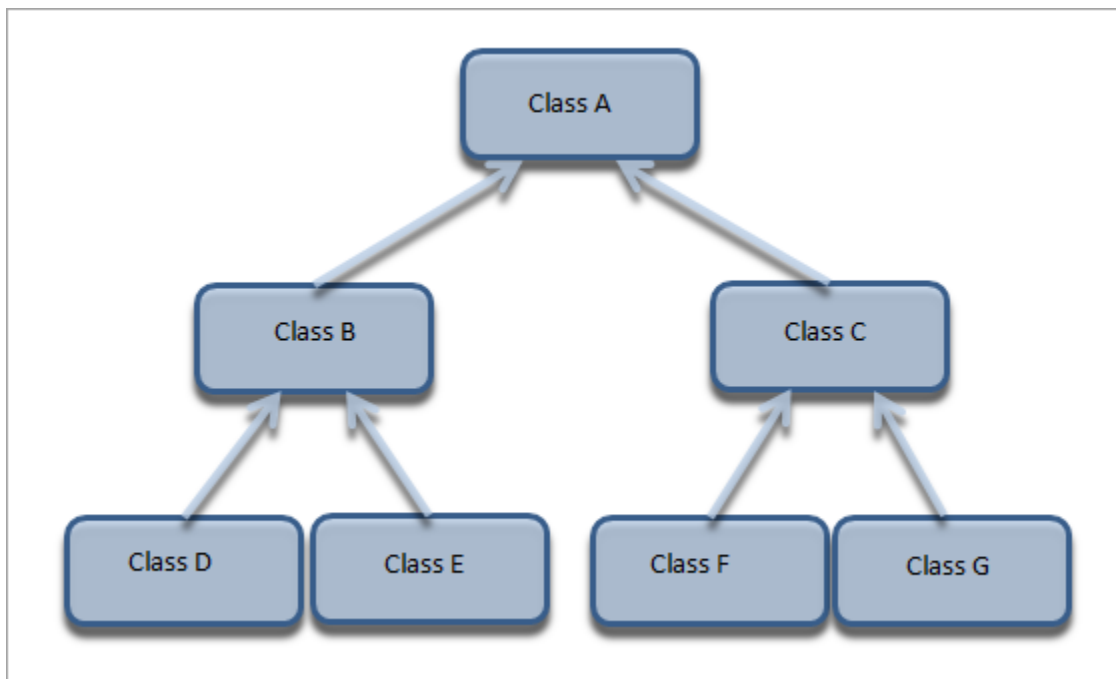


Example :- In a real-life scenario, a child inherits from their father and mother. This can be considered an example of multiple inheritance.

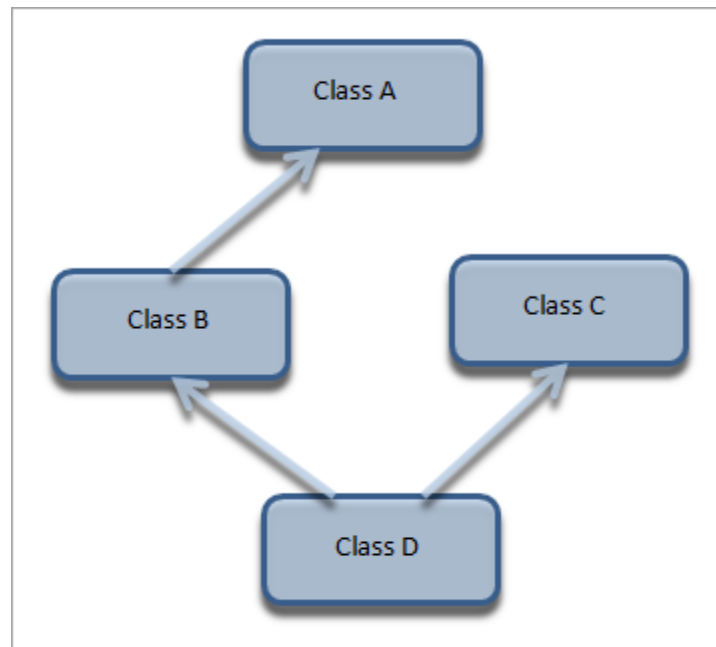
3) Multilevel Inheritance :- In multilevel inheritance, a class is derived from another derived class. This inheritance can have as many levels as long as our implementation doesn't go wayward. In the above diagram, class C is derived from Class B. Class B is in turn derived from class A.



4) Hierarchical Inheritance :- In hierarchical inheritance, more than one class inherits from a single base class as shown in the representation above. This gives it a structure of a hierarchy.



5) Hybrid Inheritance :- Hybrid inheritance is usually a combination of more than one type of inheritance. In the above representation, we have multiple inheritance (B, C, and D) and multilevel inheritance (A, B, and D) to get a hybrid inheritance.

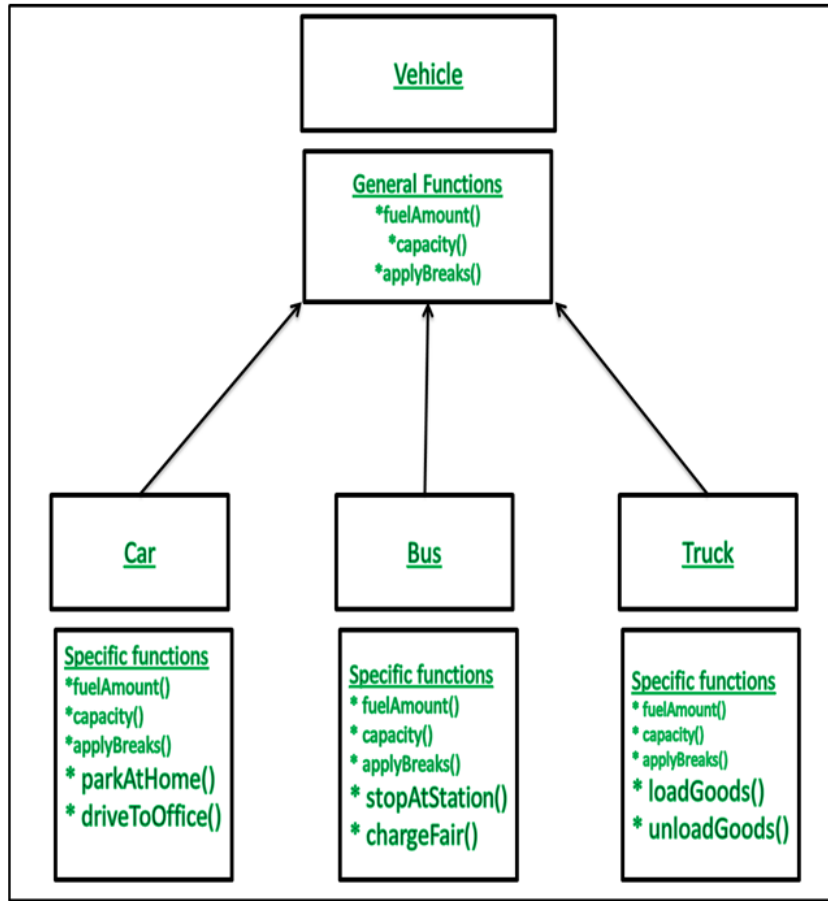


Access Specifier in Inheritance

```
1  #include<iostream>
2  using namespace std;
3  class Base {
4  public:
5      int x;
6  protected:
7      int y;
8  private:
9      int z;
10 };
11 class PublicDerived: public Base {
12     // x is public
13     // y is protected
14     // z is not accessible from PublicDerived
15 };
16 class ProtectedDerived: protected Base {
17     // x is protected
18     // y is protected
19     // z is not accessible from ProtectedDerived
20 };
21 class PrivateDerived: private Base {
22     // x is private
23     // y is private
24     // z is not accessible from PrivateDerived
25 };
```

Generalization

Up-Casting



Down-Casting

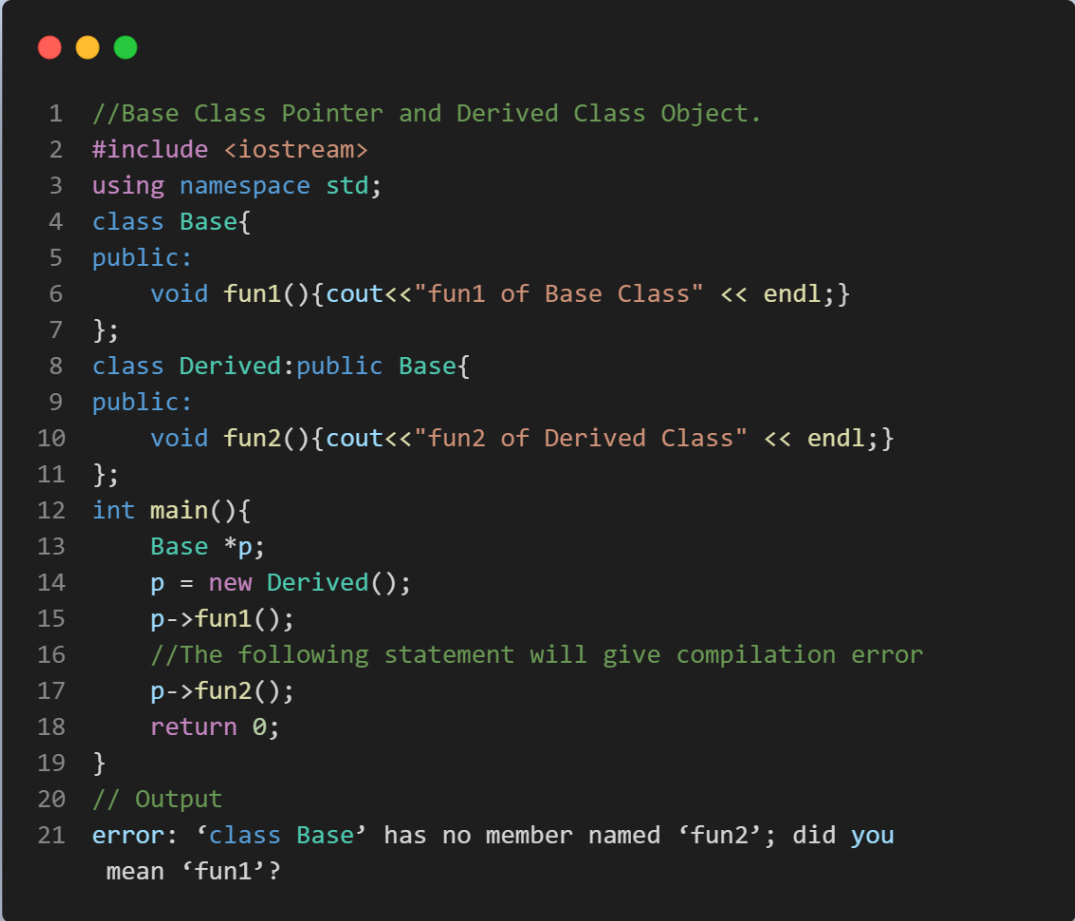
Specialization

Program to Demonstrate inheritance

```
1  // Program to Demonstrate Inheritance
2  #include <iostream>
3  using namespace std;
4  class Employee{
5  private:
6      int eid;
7      string name;
8  public:
9      Employee(int e, string n){
10         eid = e;
11         name = n;
12     }
13     int getEmployeeID(){return eid;}
14     string getName(){return name;}
15 };
16 class FulltimeEmployee : public Employee{
17 private:
18     int salary;
19 public:
20     FulltimeEmployee(int e, string n, int sal):Employee(e, n){
21         salary = sal;
22     }
23     int getSalary() { return salary; }
24 };
25 class ParttimeEmployee : public Employee{
26 private:
27     int wage;
28 public:
29     ParttimeEmployee(int e, string n, int w) : Employee(e, n){
30         wage = w;
31     }
32     int getWage() { return wage; }
33 };
34 int main(){
35     ParttimeEmployee p1(1, "John", 300);
36     FulltimeEmployee p2(2, "Raj", 5000);
37     cout<<"Salary of "<<p2.getName()<<" is "<<p2.getSalary()<<endl;
38     cout<<"Daily wage of "<<p1.getName()<<" is "<<p1.getWage()<<endl;
39 }
40
41 //Output
42 Salary of Raj is 5000
43 Daily wage of John is 300
```

Base Class Pointer and Derived Class Object in C++:-

```
Base * ptr;  
Ptr = new Derived();
```



```
1 //Base Class Pointer and Derived Class Object.  
2 #include <iostream>  
3 using namespace std;  
4 class Base{  
5 public:  
6     void fun1(){cout<<"fun1 of Base Class" << endl;}  
7 };  
8 class Derived:public Base{  
9 public:  
10     void fun2(){cout<<"fun2 of Derived Class" << endl;}  
11 };  
12 int main(){  
13     Base *p;  
14     p = new Derived();  
15     p->fun1();  
16     //The following statement will give compilation error  
17     p->fun2();  
18     return 0;  
19 }  
20 // Output  
21 error: 'class Base' has no member named 'fun2'; did you  
    mean 'fun1'?
```

Eg. Suppose there is a basic Car. A basic car means having the basic features of a car. Nothing is automated, no extra features, except for driving a car there are no extra things like there is no air conditioner, no media player, and no keyless entry. Nothing is there.

Then you have an advanced car. And an advanced car is inheriting from a basic car. So, can we have a pointer of a basic car, and to that pointer, can we assign an object of an advanced car? Yes.

Then using the basic class pointer which functions you can call? You can call only the functions which are present in the basic car. It is just like a basic car but there is an advanced car here with all the extra features in the car which you have no idea about.

```
1 //Base Class Pointer and Derived Class Object.
2 #include<iostream>
3 using namespace std;
4 class BasicCar{
5 public:
6     void Start(){cout<<"Car Started"<<endl;}
7 };
8 class AdvanceCar:public BasicCar{
9 public:
10     void PlayMusic(){cout<<"Playing Music"<<endl;}
11 };
12
13 int main()
14 {
15     AdvanceCar a;
16     BasicCar *ptr = &a;
17     ptr->Start();
18     //The following statement will throw compilation error
19     ptr->PlayMusic();
20     return 0;
21 }
22
23 //Output
24 error: 'class BasicCar' has no member named 'PlayMusic'
```

Can we create a Derived pointer assigned to the Base class object in C++?

```
Derived *p;  
p = new Base();
```

Is it possible that we have a derived class pointer and I have assigned the object of the base class? **No, not possible.** Why? Let us see the reason.

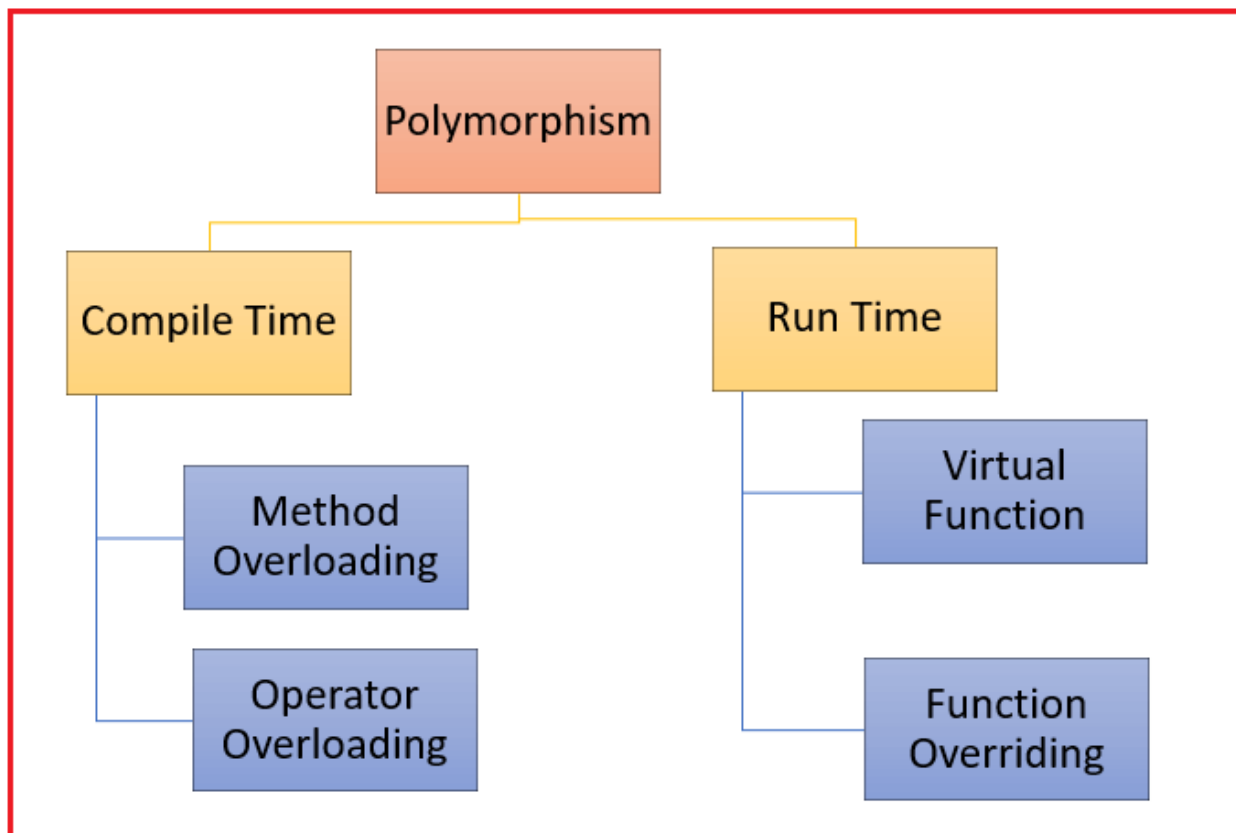
See we have a basic car. **Now if we are calling the basic car as an advanced car.** Then can we get the advanced car features in the basic car? The answer is No.

So, many of the features of our advanced car are not available in the basic car. If I think that it is an advanced car, I cannot use the features. Suppose if I tried keyless entry then it is not there, if I try to open moon roof then it's not there, if I try to start AC then it's not there.

Polymorphism

Polymorphism is considered one of the important features of Object-Oriented Programming. The word polymorphism means having many forms. In simple words, *we can define polymorphism as the ability of a message to be displayed in more than one form.*

One real-life example of polymorphism is, that a person at the same time can have different characteristics. *A man at the same time is a father, a husband, and an employee.* So, the same person possesses different behavior in different situations. This is called polymorphism.

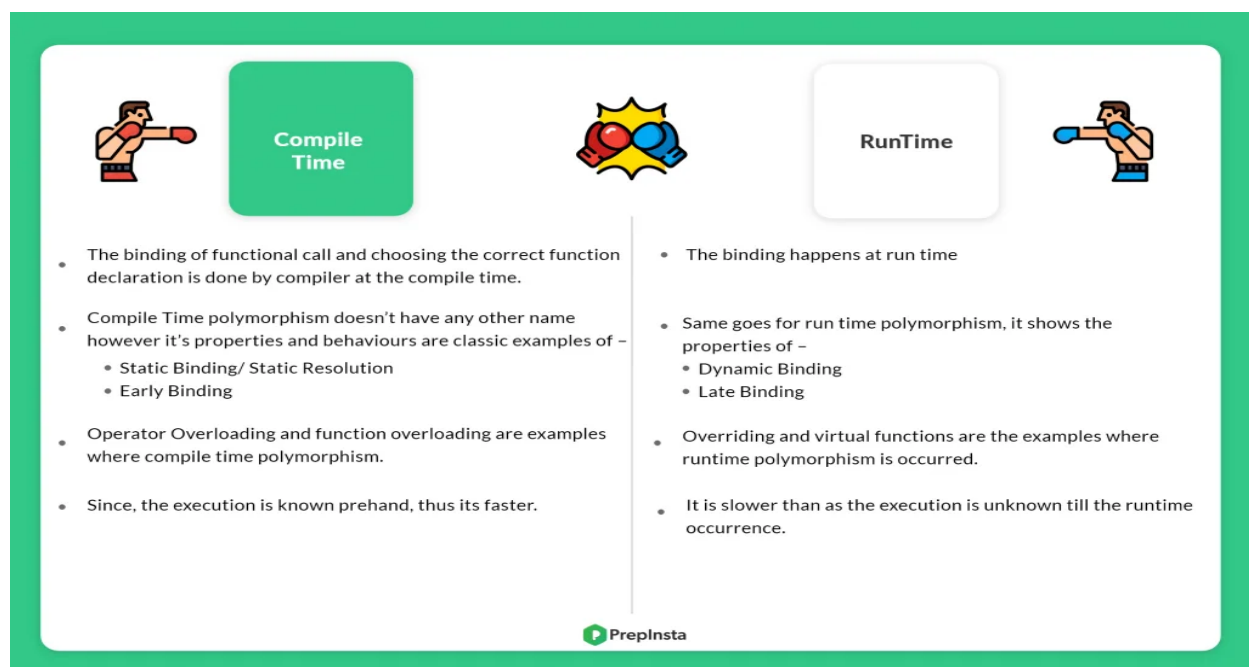


Compile Time Polymorphism in C++ :- This type of polymorphism is achieved by function overloading or operator overloading. The overloaded functions are invoked by matching the type and number of arguments.

The information is present during compile-time. This means the C++ compiler will select the right function at compile time. Which is also known as static binding or early binding.

Runtime Polymorphism in C++ :- Run time polymorphism is achieved when the object's method is invoked at the run time instead of compile time. It is achieved by method overriding which is also known as dynamic binding or late binding.

This type of polymorphism is achieved by Function Overriding. This happens when an object's method is invoked/called during runtime rather than during compile time.



Function overriding -

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  class Base{
4  public:
5      void Display(){
6          cout << "Base Class Display Function";
7      }
8  };
9  class Derived:public Base{
10 public:
11     void Display(){
12         cout << "Derived Class Display Function";
13     }
14 };
15 int main(){
16     Derived d;
17     d.Display();
18     return 0;
19 }
20 //Output
21 Derived Class Display Function
```

Key Points about Function overriding :-

- Redefining a function of a base class in the derived class called function overriding in c++.
- Function overriding is used for achieving runtime polymorphism.
- The Signature and Prototype of an overrides function must be exactly the same as the base function.

When do we need to override a function in C++?

If the superclass function logic is not fulfilling the sub-class business requirements, then the subclass needs to override that function with the required business logic. Usually, in most real-time applications, the superclass functions are implemented with generic logic which is common for all the next-level sub-classes.

Note :- If a function in the sub-class contains the same signature as the superclass non-private function, then the subclass function is treated as the **overriding function** and the superclass function is treated as the **overridden function**.

Virtual Function in C++ :- A virtual function in C++ is a member function that is declared within a base class using the virtual keyword and is re-defined by a derived class. When we refer to a **derived class object using the base class pointer** or base class reference variable, and when we can call a virtual function, then it will execute the function from the derived class.

So, Virtual Functions in C++ ensure that the correct function is called for an object, regardless of the expression used to make the function call.

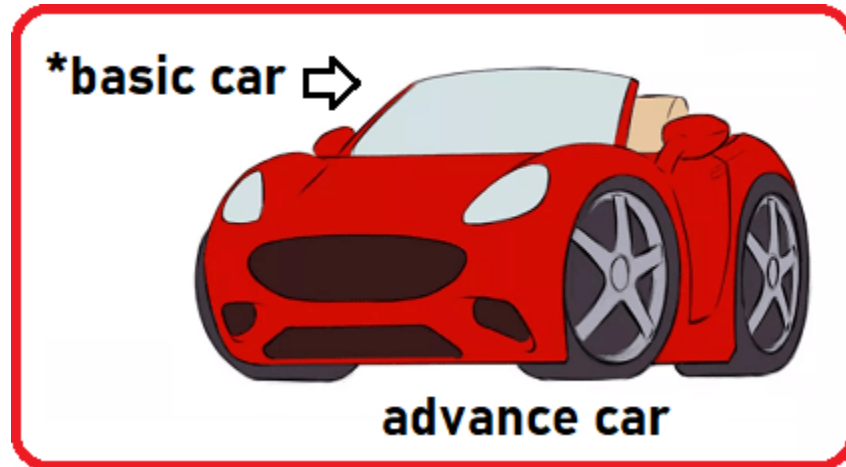


```
1  #include<bits/stdc++.h>
2  using namespace std;
3  class Base{
4  public:
5      void Display(){
6          cout << "Display of Base" << endl;
7      }
8  };
9  class Derived:public Base{
10     public: void Display(){
11         cout << "Display of Derived " << endl;
12     }
13 };
14 int main(){
15     Base *p = new Derived ();
16     p->Display ();
17 }
18 //Output
19 Display of Base
```



```
1  #include<bits/stdc++.h>
2  using namespace std;
3  class Base{
4  public:
5      virtual void Display(){
6          cout << "Display of Base" << endl;
7      }
8  };
9  class Derived:public Base{
10     public: void Display(){
11         cout << "Display of Derived " << endl;
12     }
13 };
14 int main(){
15     Base *p = new Derived ();
16     p->Display ();
17 }
18 //Output
19 Display of Derived
```

Real-Time Example to Understand the need for Virtual Function :



Pointer is the basic car but the actual object is the advanced car. So, I am thinking of it as a basic car. How can I drive? How it will run? Is it will run like what I am thinking or run like an advanced car? It will run like an advanced car because I am pointing at the object of an advanced car.

So, when you call a function that is present in the base case as well as the derived class, whose function must be called? Based on the object the function must be called. Not based on the pointer.

See it is just like if you saw a donkey and say it's a horse, will it run like a horse? You are thinking of it like a horse but that's a donkey. It cannot run like a horse. In the same way, here the basic car is the reference and you are pointing at the advanced car object. So, the object is of the derived class and the pointer is of the base class.

When we call a function then the base class function is called. But logically it is wrong. Whose function must be called? The function must be called based on the object.

Key points about virtual functions :-

1. Virtual functions are used for achieving polymorphism
2. The base class can have virtual functions
3. Virtual functions **can be** overrides in the derived class
4. Pure virtual functions **must be** overrides by the derived class

Rules for virtual functions :-

1. Virtual functions in C++ cannot be static.
2. A virtual function in C++ can also be a friend function of another class.
3. The prototype/Signature of virtual functions should be the same in the base as well as the derived class.
4. In C++, a class may have a virtual destructor but it cannot have a virtual constructor.

Limitation of virtual functions :-

- Slower
- Difficult to debug

Runtime Polymorphism :-

```
1  #include<bits/stdc++.h>
2  using namespace std;
3  class Car{
4  public:
5      void Start(){cout<<"Car Started"<<endl;}
6      void Stop(){cout<<"Car Stopped"<<endl;}
7  };
8  class Innova:public Car{
9  public:
10     void Start(){cout<<"Innova Started"<<endl;}
11     void Stop(){cout<<"Innova Stopped"<<endl;}
12 };
13
14 class Swift:public Car{
15 public:
16     void Start(){cout<<"Swift Started"<<endl;}
17     void Stop(){cout<<"Swift Stopped"<<endl;}
18 };
19 int main(){
20     Car *c = new Innova();
21     c->Start();
22     c->Stop();
23     c = new Swift();
24     c->Start();
25     c->Stop();
26     return 0;
27 }
28 //Output
29 Car Started
30 Car Stopped
31 Car Started
32 Car Stopped
```

Now, suppose we want the derived class function should be called then we need to make the base class functions as virtual functions. This means if we want the Innova class Start function should be called then we have to make the Car class Start function virtual.

```
1  #include<bits/stdc++.h>
2  using namespace std;
3  class Car{
4  public:
5      virtual void Start(){cout<<"Car Started"<<endl;}
6      virtual void Stop(){cout<<"Car Stopped"<<endl;}
7  };
8  class Innova:public Car{
9  public:
10     void Start(){cout<<"Innova Started"<<endl;}
11     void Stop(){cout<<"Innova Stopped"<<endl;}
12 };
13
14 class Swift:public Car{
15 public:
16     void Start(){cout<<"Swift Started"<<endl;}
17     void Stop(){cout<<"Swift Stopped"<<endl;}
18 };
19 int main(){
20     Car *c = new Innova();
21     c->Start();
22     c->Stop();
23     c = new Swift();
24     c->Start();
25     c->Stop();
26     return 0;
27 }
28 //Output
29 Innova Started
30 Innova Stopped
31 Swift Started
32 Swift Stopped
```

Key Points of Runtime Polymorphism in C++:

- Same name different actions
- Runtime Polymorphism is achieved using function overriding
- Virtual functions are abstract functions of the base class
- The derived class must override the virtual functions
- A base class pointer pointing to a derived class object and an override function is called

Abstract class in C++

An abstract class in C++ has at least one pure virtual function by definition. In other words, a function that has no definition and these classes cannot be instantiated. The abstract class's child classes must provide body to the pure virtual function; otherwise, the child class would become an abstract class in its own right.

What is the purpose of the Pure Virtual function?

The purpose of a pure virtual function is to achieve polymorphism. We want the derived classes to override this function. So, it becomes mandatory for derived classes to override the pure virtual function. It means the base class is governing or giving a command to the child classes that you must override the pure virtual functions. So, it is defining an interface.

If a class is having a pure virtual function, then that class is called an **Abstract Class in C++**.

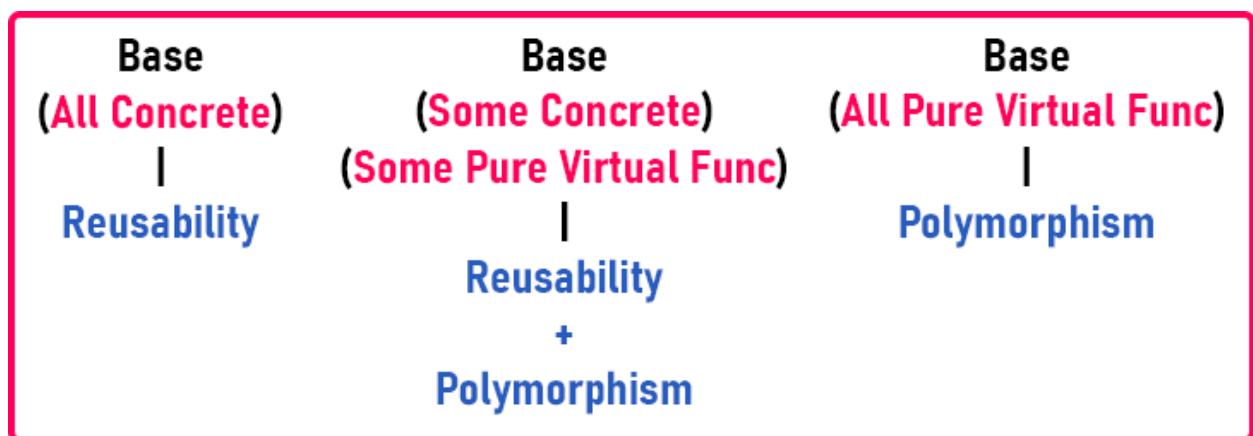
Can we create the object of Abstract Class in C++? No.

Can we create the pointer of the abstract class? Yes.



What is the purpose of inheritance?

- Two things, one is reusability. The derived class gets the function from the base class.
- The second thing to achieve is polymorphism.



Key Points of Abstract class in C++:

- The class having a pure virtual function is an abstract class.
- An **Abstract class** can have concrete also.
- The object of abstract class cannot be created
- A derived class must override pure virtual function, otherwise, it will also become an abstract class.
- The pointer of the abstract class can be created.
- The pointer of the abstract class can hold the object of the derived class.
- Abstract classes are used for achieving polymorphism
- Abstract with some concrete and some pure virtual functions.



```
1  #include<bits/stdc++.h>
2  using namespace std;
3  class Shape{
4  public:
5      virtual float Area() = 0;
6      virtual float Perimeter() = 0;
7  };
8  class Rectangle:public Shape{
9  private:
10     float length;
11     float breadth;
12 public:
13     Rectangle(int l = 1, int b = 1){
14         length = 1;
15         breadth = b;
16     }
17     float Area(){return length * breadth;}
18     float Perimeter(){return 2 * (length + breadth);}
19 };
20
21 class Circle:public Shape{
22 private:
23     float radius;
24 public:
25     Circle(float r){radius = r;}
26     float Area(){return 3.1425 * radius * radius;}
27     float Perimeter(){return 2 * 3.1425 * radius;}
28 };
29
30 int main(){
31     Shape *s = new Rectangle(10, 5);
32     cout<<"Area of Rectangle "<<s->Area()<<endl;
33     cout<<"Perimeter of Rectangle "<<s->Perimeter()<<endl;
34     s = new Circle(10);
35     cout<<"Area of Circle "<<s->Area()<<endl;
36     cout<<"Perimeter of Circle "<<s->Perimeter()<<endl;
37 }
38
39
```

```
≡ output.txt X
≡ output.txt
1  Area of Rectangle 5
2  Perimeter of Rectangle 12
3  Area of Circle 314.25
4  Perimeter of Circle 62.85
```

Friend Function and Friend Classes in C++

A friend function in C++ is defined as a function that can access private, protected, and public members of a class. A friend function can be a member of another class or can be a global function in C++.

Note :- A friend function of a class is defined outside that class scope but it has the right to access all private, protected, and public members of the class. Even though the prototypes for friend functions appear in the class definition, friends are not member functions.

Characteristics of a Friend Function in C++:

- It cannot be called using the object as it is not in the scope of that class.
- It can be invoked like a normal function without using the object.
- It cannot access the member names directly and has to use an object name and dot membership operator with the member's name.
- It can be declared either in the private or the public part.
- The function is not in the scope of the class of which it has been declared as a friend



```
1  #include<bits/stdc++.h>
2  using namespace std;
3  class Test{
4  private:
5      int x;
6  protected:
7      int y;
8  public:
9      int z;
10     friend void Fun();
11 };
12 void Fun(){
13     Test t;
14     t.x = 10, t.y = 20, t.z = 30;
15     cout << " X : " << t.x << endl;
16     cout << " Y : " << t.y << endl;
17     cout << " Z : " << t.z << endl;
18 }
19 int main(){
20     Fun();
21     return 0;
22 }
23 //Output
24 X : 10
25 Y : 20
26 Z : 30
```

Friend Classes in C++:

In C++, a friend class can access private, protected, and public members of another class in which it is declared a friend. It is sometimes useful to allow a particular class to access private members of other classes.

Instead of writing the name before the My Class definition, we can also write `friend class Your;`

```
1  #include<bits/stdc++.h>
2  using namespace std;
3  class My{
4  private:
5      int x;
6  protected:
7      int y;
8  public:
9      int z;
10     friend class Your;
11 };
12 class Your{
13 public:
14     My m;
15     void Fun(){
16         m.x = 10;
17         m.y = 20;
18         m.z = 30;
19         cout << "X = " << m.x << endl;
20         cout << "Y = " << m.y << endl;
21         cout << "Z = " << m.z << endl;
22     }
23 };
24 int main(){
25     Your obj;
26     obj.Fun ();
27     return 0;
28 }
```

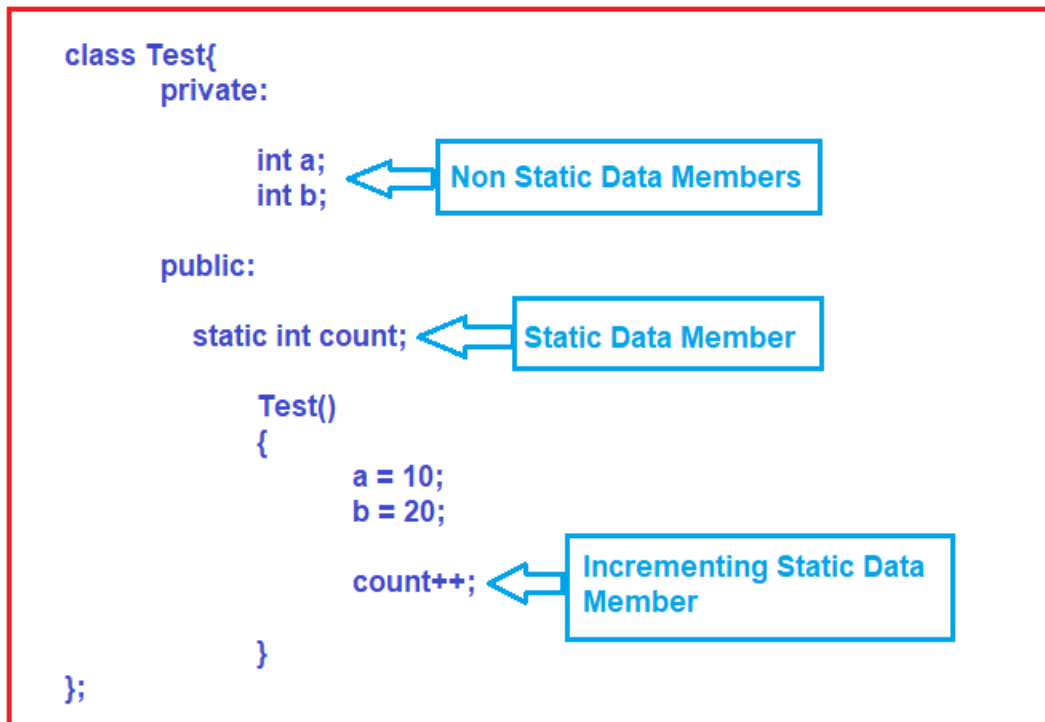
Key Points of Friend Functions and Classes in C++ :-

- Friend functions are global functions
- They can access private, protected, and public members of a class upon their objects
- A class can be declared as a friend of another class
- All the functions of the friend class can access private and protected members of other classes.
- **Friendship is not mutual.** If class A is a friend of B, then B doesn't become a friend of A automatically.
- **Friendship is not inherited.**
- Friends should be used only for a limited purpose.
- Too many functions or external classes are declared as friends of a class with protected or private data, which is against the principle of encapsulation in object-oriented programming.

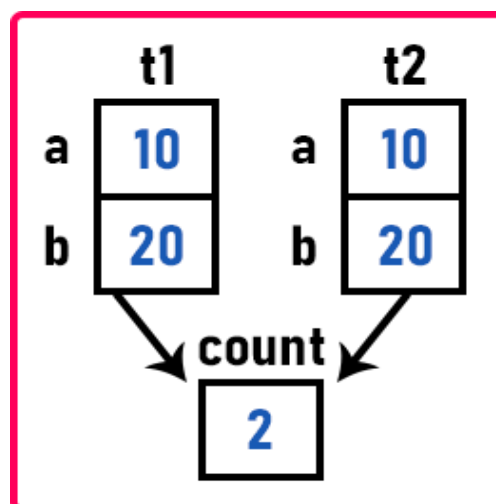
Static Data Members in C++ :-

We can define class members' static using the static keyword in C++. When we declare a member of a class as static **it means no matter how many objects of the class are created, there is only one copy of the static member available throughout the program. So static variables or static data members of a class belong to a class. That doesn't belong to an object.**

It means when we make a variable as static, that variable will occupy memory only once. And all the objects can share it.



One more thing we have to address syntactically. When we have a static variable inside the class, **you must have declared it outside again**. So, when we declare the class variable outside the class, then it will become a global variable but we want it to be accessible only inside the `Test` class. So, we can use the scope resolution operator to make it accessible only inside the `Test` class.





```
1  #include<bits/stdc++.h>
2  using namespace std;
3  class Test{
4  private:
5      int a;
6      int b;
7  public:
8      static int count;
9      Test(){
10         a = 10, b = 20;
11         count++;
12     }
13 };
14 int Test::count = 0;
15 int main(){
16     Test t1, t2;
17     cout<<t1.count<<endl;
18     cout<<t2.count<<endl;
19     cout<<Test::count<<endl;
20     return 0;
21 }
22 //Output
23 2
24 2
25 2
```

Static Member Functions in C++ :- By declaring a member function as static in C++, we make it independent of any particular object of the class. A static member function can be called even if no objects of the class exist and the static functions are accessed using the class name and the scope resolution operator (::) and even we can also access them using objects of the class.

Note :- the important thing is Static Member Functions in C++ can access only static data members of a class. They cannot access non-static data members. So static member functions also belong to a class.

```
1  #include<bits/stdc++.h>
2  using namespace std;
3  class Test{
4  public:
5      int a;
6      static int count;
7      Test(){
8          a = 10;
9          count++;
10     }
11     static int getCount(){
12         return count;
13     }
14 };
15 int Test::count = 0;
16 int main(){
17     cout<<"Calling count without object : "<<Test::count<<endl;
18     cout<<"Calling getCount without object : "<<Test::getCount()<<endl;
19     Test t1;
20     cout<<"Calling count with object t1 : "<<t1.count<<endl;
21     cout<<"Calling getCount with object t1 : "<<t1.getCount()<<endl;
22 }
23 //Output
24 Calling count without object : 0
25 Calling getCount without object : 0
26 Calling count with object t1 : 1
27 Calling getCount with object t1 : 1
```

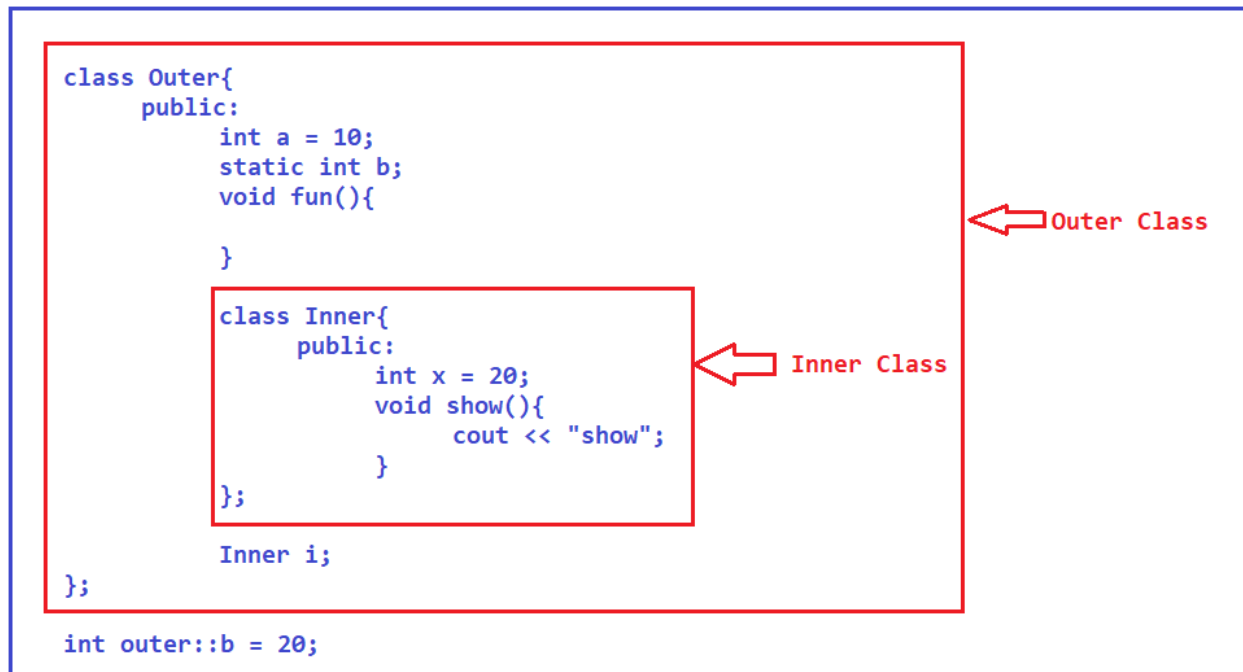
Key Points on Static Members in C++:

- Static data members are members of a class.
- Only one instance of static members is created for the entire class and shared by all objects of that class, no matter how many objects are created.
- It is initialized before any object of this class is created, even before the main starts.
- They can be accessed directly using the class name.
- Static member functions are functions of a class, they can be called using the class name, without creating the object of a class.
- They can access only static data members of a class; they cannot access non-static members of a class.
- It is visible only within the class, but its lifetime is the entire program.

Inner Class in C++ :-

Writing a class inside another class is called an **inner class**. It is useful only within that class. **If you are writing a class that is not useful everywhere or limited to one class.** Then you can write a class inside of another one.

So, it is just like reducing the complexity of a bigger class, we can write smaller classes inside.



Note :- Can we create this object before the declaration of the Inner class? No, it must be done after the definition.

- Inner class can access the members of the Outer class if they are static.
- Can the Outer class create the object of the Inner class? Yes, it can. Now using that object can it access all the members of a class
- We can access only those members which are public. We cannot access private and protected members of the Inner class.

When to use Inner Classes in C++?

```
class LinkedList{
    class Node{
        int data;
        Node *next;
    };
    Node *Head;
    ----
    ----
};
```



```
1  #include<bits/stdc++.h>
2  using namespace std;
3  class Outer{
4  public:
5      void Fun (){
6          i.Display();
7      }
8      class Inner{
9      public:
10         void Display(){
11             cout<<"Display of Inner"<<endl;
12         }
13     };
14     Inner i;
15 };
16 int main(){
17     Outer::Inner i;
18     i.Display();
19     return 0;
20 }
21 //Output
22 Display of Inner
```

Exception Handling in C++ :-

There are 3 types of errors that are occurred in programming:-

- Syntax Error
- Logical Error
- Runtime Error

Syntax Error :- What is a Syntax error? While typing the program, if the programmer mistypes something or did not write something properly then there is an error known as Syntax Error. So, who will identify this error? The compiler will identify this error.

Logical Error :- Suppose you wanted to do something and so wrote the procedure or function or some code but when you run the program the results are different i.e. not as expected.

Then is there any tool for solving this type of problem? Yes, that is a debugger. The debugger will help you to run the program line by line or statement by statement, so in each statement, you can see what is happening and wherever it is going wrong and you can catch it and you can solve that problem.

Note :- The most important thing is that these two types of errors (Syntax Error And Logical Error) are faced by programmers.

Runtime Error :- Suppose you are running some business and you wanted me to develop an application for your business. So, I developed an application and that application is perfect. It is neither having any syntax errors nor having any logical errors. Then I delivered the software to you and you are a user and using that application for your business.

So, the user is going to use the application and while using the application he/she must use it properly. **If he/she mishandled the application then it may cause errors during the runtime of a program.** It's not development time. **At the development time, as a programmer, I have faced logical and syntax errors that I have already removed.** Now it is perfect. There are no errors. And you are using the program. So, at runtime, you are facing errors.

What Could Be the Cause of Error During Runtime?

- First of the reasons for runtime errors is bad input.
- Second, the program that I have given you requires an internet connection but if you are not providing an internet connection then the program cannot continue.
- Third, the program that I have given to you needs some files on your computer system but you have deleted those files or my program is using a printer but you do not have a printer. So, these are the list of problems. **These are nothing but the unavailability of resources.**

All the things that are outside the program or not in the control of the program. Those are in the control of the user.

What happens in the case of runtime errors?

In case of runtime errors, the program crashes or program stops abnormally without completing its execution. Suddenly the program will stop, that's how the runtime errors are dangerous for the user. So, who is responsible? The user is responsible because as a developer I did my job perfectly and the user is not providing proper resources or proper input so the program is failing. Now let us give you some real-life examples.

What can we call these runtime errors?

These runtime errors are called exceptions. Why we are using the term exception? Suppose I have given you a perfect program and it will always run perfectly except in some conditions. Those conditions are giving a bad input or not providing resources. So, the program will not run in those cases. That's why we call these exceptional cases or exceptions.

What are Exceptions?

An exception is nothing but a situation in which we get the runtime error. Let us see in automobile engineering. If there is no fuel in the car then the car will indicate that there is less fuel or it is in a reserved condition so you must find the closest fuel station or gas station.

In the same way, the programs should also behave like that. If there is bad input then the program should not stop suddenly. It should ask the user that **'this input is not proper please enter another type of Input and try again'**. If there is no internet connection then the program should not stop suddenly but it should give a prompt to the user that **"we found that there is no internet connection please connect to the internet and run it again"**. So, **the idea is the program should not terminate under such a situation but guide the user to solve the problem.**

Because who is there to resolve it? The programmer will not come to the client-side and check for the errors and remove them. The program itself can give a proper and meaningful message to the user so that users can remove this type of problem.

Note :- Giving a proper message to the user and informing them about the exact problem. And also providing him guidance to solve that problem, that's it. This is the objective of exception handling.

How to Implement Exception Handling in C++?

```
try {  
    // Block of code to try  
    throw exception; // Throw an exception when a problem arise  
}  
catch () {  
    // Block of code to handle errors  
}
```

Inside the try block, if there is any error then it will jump to the catch block and execute the statements inside the catch block.

- **try:** It represents a block of code that can throw an exception.
- **catch:** It represents a block of code that is executed when a particular exception is thrown.
- **throw:** It is used to throw an exception inside a try block

```
1  #include<iostream>
2  using namespace std;
3  int Division (int a, int b) throw (int){
4      if (b == 0)
5          throw 1;
6      return a / b;
7  }
8  int main(){
9      int x, y, z;
10     cout << "Enter two Numbers:";
11     cin >> x >> y;
12     try{
13         z = Division (x, y);
14         cout << z << endl;
15     }
16     catch (int e){
17         cout << "Division by zero " << e << endl;
18     }
19     cout << "Bye" << endl;
20 }
21 //Output
22 Enter two Numbers:20 0
23 Division by zero 1
24 Bye
```

Throwing Exceptions from C++ constructors :-

An exception should be thrown from a C++ constructor whenever an object cannot be properly constructed or initialized. Since there is no way to recover from failed object construction, an exception should be thrown in such cases.

Since C++ constructors do not have a return type, it is not possible to use return in the codes. Therefore, the best practice is for constructors to throw an exception to signal failure.

```
1  #include<iostream>
2  using namespace std;
3  class Rectangle{
4  private:
5      int length;
6      int breadth;
7  public:
8      Rectangle(int l, int b){
9          if (l < 0 || b < 0){
10             throw 1;
11         }
12         else{
13             length = l;
14             breadth = b;
15         }
16     }
17     void Display()
18     {
19         cout<<"Length: "<<length<<" Breadth: "<<breadth;
20     }
21 };
22 int main(){
23     try{
24         Rectangle r1(10, -5);
25         r1.Display();
26     }
27     catch (int num){
28         cout<<"Rectangle Object Creation Failed";
29     }
30 }
31 //Output
32 Rectangle Object Creation Failed
```

Key Points regarding exception handling :-

- An exception in C++ is thrown by using the throw keyword from inside the try block. The throw keyword allows the programmer to define custom exceptions.
- Multiple handlers (catch expressions) can be chained - each one with a different exception type. Only the handler whose argument type matches the exception type in the throw statement is executed.
- C++ does not require a finally block to make sure resources are released if an exception occurs.

Using Multiple catch blocks in C++ :-

```
1  #include<iostream>
2  #include<conio.h>
3  using namespace std;
4  int main(){
5      int arr[3] ={-1, 2};
6      for (int i = 0; i < 2; i++){
7          int num = arr[i];
8          try{
9              if (num > 0) throw 1;
10             else throw 'a';
11         }
12         catch (int ex){
13             cout << "Integer Exception" << endl;
14         }
15         catch (char ex){
16             cout << "Character Exception" << endl;
17         }
18     }
19     return 0;
20 }
```

Generic Catch Block (Catch - All) in C++ :-

The following example contains a generic catch block to catch any uncaught errors/exceptions. catch(...) block takes care of all types of exceptions.

```
1  #include <iostream>
2  #include<conio.h>
3  using namespace std;
4  int main(){
5      int arr[3] = { -1, 2, 5 };
6      for (int i = 0; i < 3; i++){
7          int num = arr[i];
8          try{
9              if(num == -1) throw 1;
10             else if(num == 2) throw 'a';
11             else throw "Generic";
12         }
13         catch(int ex){
14             cout<<"Integer Exception"<<endl;
15         }
16         catch (char ex){
17             cout<<"Character Exception"<<endl;
18         }
19         //Generic catch block
20         catch (...){ // to catch anytime of exceptions
21             cout<<"Generic Exception"<<endl;
22         }
23     }
24     return 0;
25 }
```

Note : If we write catch-all first, then all the exceptions will be handled here only. The lower catch blocks will never be executed. So, the catch-all block must be the last block.

Throwing User-Defined Type in C++ :-

```
1  #include<iostream>
2  #include<conio.h>
3  using namespace std;
4  class myExp{
5      //Nothing is here
6  };
7  int main (){
8      int num = 2;
9      try{
10         if (num == 1) throw 1;
11         else if (num == 2) throw myExp();
12         else if (num == 3) throw "Unknown Exception";
13         else cout<<"Value"<<num<<endl;
14     }
15     catch(int ex){
16         cout<<"Integer Exception"<<endl;
17     }
18     catch (myExp e){
19         cout<<"myExp Exception"<<endl;
20     }
21     catch (...){
22         cout<<"Unknown Exception"<<endl;
23     }
24     return 0;
25 }
26 int main (){
27     int num = 2;
28     try{
29         if (num == 1) throw 1;
30         else if (num == 2) throw myExp();
31         else if (num == 3) throw "Unknown Exception";
32         else cout<<"Value"<<num<<endl;
33     }
34     catch(int ex){
35         cout<<"Integer Exception"<<endl;
36     }
37     catch(myExp e){
38         cout<<"myExp Exception"<<endl;
39     }
40     catch(...){
41         cout<<"Unknown Exception"<<endl;
42     }
43     return 0;
44 }
```

Can we have a Try Block inside a Try block?

Yes, we can have a try block inside another try block in C++. The following diagram shows the syntax of writing nested try blocks in C++.

```
try{  
    --- 1  
    --- 2  
    try{  
        --- 1  
        --- 2  
    }  
    catch(){  
    }  
}  
catch(){  
}
```

Example to Understand Try-Catch Blocks in C++:

```
class MyExp1{};
```

```
class MyExp2 : public MyExp1{};
```

So, we have these two classes. And MyExp2 is publicly inherited from the MyExp1 class. So, MyExp1 is a parent class and MyExp2 is a child class.

```
try{  
    ---  
    ---  
    ---  
}  
catch(MyExp1 e){  
}  
catch(MyExp2 e){  
}
```

```
try{  
    ---  
    ---  
    ---  
}  
catch(MyExp2 e){  
}  
catch(MyExp1 e){  
}
```

As you can see, we have written catch blocks for both types of exceptions. So, is this the correct format of catch block? No. We have written the catch block for the parent class first and then for the child class. **We must write catch blocks for the child class first and then for the parent class as shown in the below image.**

Inheriting Exception Class from Built-in Exception Class :-

If you are throwing your own class exception then better extend your class from the built-in exception class in C++ as follows. **But it is not mandatory.**

```
class MyException: public exception {  
};
```

Overriding the exception class what method in C++ :-

After inheriting our custom exception class from the built-in exception class, do we have to override anything? **Yes, we can override one method which is "what" method as follows. But this is not mandatory as in our previous example we have not overridden the what method and it is working fine.**

```
class MyException : public exception{
    public:
        char * what(){
            return "My Custom Exception";
        }
};
```

How to make the function throws something in C++?

```
int Division(int a, int b) throw (MyException)
{
    if (b == 0)
        throw MyException();
    return a / b;
}
```

```
int Division(int a, int b) throw (int)
{
    if (b == 0)
        throw 1;
    return a / b;
}
```

So, whatever the type of value you are throwing, you can mention that in the brackets. **And if there are more values then you can mention them with commas as follows:**

```
int Division(int a, int b) throw (int, MyException)
{
    if (b == 0)
        throw 1;
    return a / b;
}
```

```

1  #include<iostream>
2  using namespace std;
3  #include <exception>
4  class MyException:public exception{
5  public:
6      char * what(){
7          return "My Custom Exception";
8      }
9  };
10 int Division(int a, int b) throw (int, MyException){
11     if (b == 0) throw 1;
12     if (b == 1) throw MyException();
13     return a / b;
14 }
15 int main(){
16     int x = 10, y = 1, z;
17     try{
18         z = Division(x, y);
19         cout<<z<<endl;
20     }
21     catch(int x){
22         cout<<"Division By Zero Error"<<endl;
23     }
24     catch (MyException ME){
25         cout<<"Division By One Error"<<endl;
26         cout<<ME.what ()<<endl;
27     }
28     cout<<"End of the Program"<<endl;
29 }

```

Can we use cout instead of throw?

If you use cout, the user will know the error. **And if you use throw, it will inform the calling function about the error.**

Are return and throw the same?

Return is for returning results. The throw is for reporting an error. If you change their roles then the roles of Try and Catch will also change.

Template in C++

Template in C++ allows us to define Generic Functions and Generic Classes. That means using a Template we can implement Generic Programming in C++. They are going to work with a variety of data types. Templates in C++ can be represented in two ways. They are as follows.

- Function templates
- Class templates

```
int max(int a, int b){  
    return a > b ? a : b;  
}
```

```
T max(T a, T b){  
    return a > b ? a : b;  
}
```

```
1  #include<iostream>  
2  using namespace std;  
3  template <class T, class R>  
4  void Add (T x, R y){  
5      cout << x + y << endl;  
6  }  
7  int main(){  
8      //Integer and Integer  
9      Add (4, 24);  
10     //Float and Float  
11     Add (25.7f, 67.6f);  
12     //Integer and double  
13     Add (14, 25.5);  
14     //Float and Integer  
15     Add (25.7f, 45);  
16     return 0;  
17 }  
18 //Output  
19 28  
20 93.3  
21 39.5  
22 70.7
```

```

1  #include <iostream>
2  using namespace std;
3  template<class T>
4  class Stack{
5      private:
6          T * stk;
7          int top, size;
8      public:
9          Stack (int sz){
10             size = sz;
11             top = -1;
12             stk = new T[size];
13         }
14         void Push(T x);
15         T Pop();
16     };
17     template<class T>
18     void Stack<T>::Push(T x){
19         if (top == size - 1)
20             cout<<"Stack is Full"<<endl;
21         else{
22             top++;
23             stk[top] = x;
24             cout<<x <<" Added to Stack"<<endl;
25         }
26     }
27     template<class T>
28     T Stack<T>::Pop(){
29         T x = 0;
30         if (top == -1)
31             cout<<"Stack is Empty"<<endl;
32         else{
33             x = stk[top];
34             top--;
35             cout<<x<<" Removed from Stack"<<endl;
36         }
37         return x;
38     }
39     int main(){
40         //Stack of Integer
41         Stack<float> stack1(10);
42         stack1.Push(10), stack1.Push(23),stack1.Push(33);
43         stack1.Pop();
44         //Stack of double
45         Stack<double> stack2(10);
46         stack2.Push(10.5), stack2.Push(23.7), stack2.Push(33.8);
47         stack2.Pop();
48         return 0;
49     }

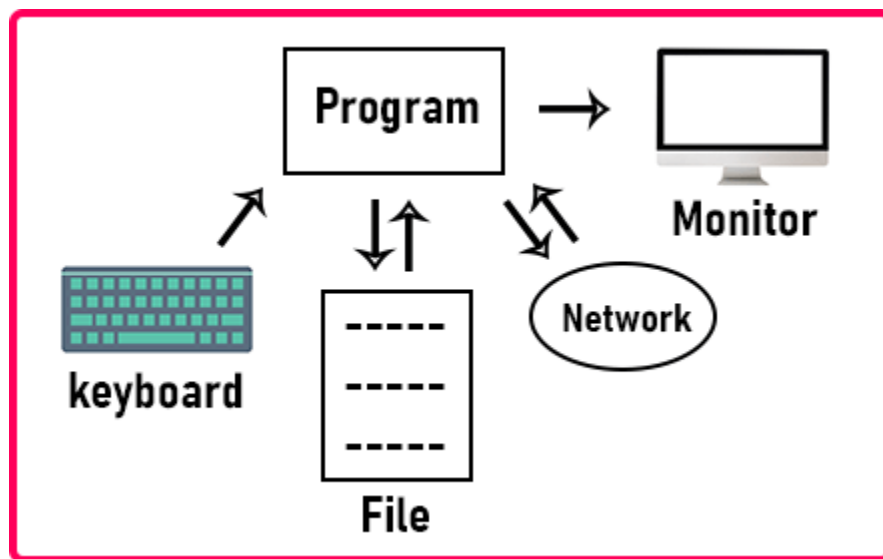
```

File Handling in C++

Before learning File Handling in C++, let us first learn about Streams in C++. **A stream is a flow of data or a flow of characters.**

Streams are a way to perform input and output operations in C++.

They are objects that allow you to read or write a sequence of characters, **typically from or to a file**. Streams are used to perform these operations in a flexible and efficient manner.



If we have a program, then the program may be getting the data from the keyboard or program may be sending the data to the monitor or the program may be accessing the data from an external file.

istream: Represents an input stream and is used to read data from an input source (e.g., a file or the console).

ostream: Represents an output stream and is used to write data to an output destination (e.g., a file or the console).

iostream: Represents an input/output stream and can be used to both read and write data.

Now similarly, for file access also, there are classes available that are ifstream for input stream and ofstream for the output stream.

For input stream from the keyboard, we have already a built-in object present in iostream header file that is cin and also an extraction operator that is ">>". So directly we can use cin with extraction operator. **Cin is an object of istream class.** And we have another object that is cout that we commonly used for displaying the text on the screen. **cout is an object of ostream.** cout is used with the insertion operator "<<".

```
#include <fstream>
int main(){
    ofstream outfile("my.txt");
}
```

So, this object outfile will associate with the file my.txt. Now whatever you will write into this outfile object it will get dropped into that file. **It's like you have connected a pipe from your program to that file on the disk. So, whatever you drop in the outfile it will get stored in the file.**

Next thing, when we were opening this file through outfile object, the file should be already existing. **If it is existing then outfile will open the file and if it is not existing then it will create a new file with the name my.txt.**

Next point, if the already file is existing and it is **having some content then what will happen to that content?** The outfile will truncate that content or remove the content. Suppose my.txt has some content, my.txt: 100 200 300

So, we have these numbers in the my.txt file. **These values will be removed and fresh content will be written.** If you want to append the new content after the old content then there is a mode available that is **ios::app**. 'app' stands for append.

So like this mode, there are two modes available that are **ios::app** and **ios::trunc**. 'trunc' stands for truncate.

After writing the content, you must close the file by writing the following statement. `outfile.close();`

This statement will close the stream and the file will be free from the program. It is important to close the file.

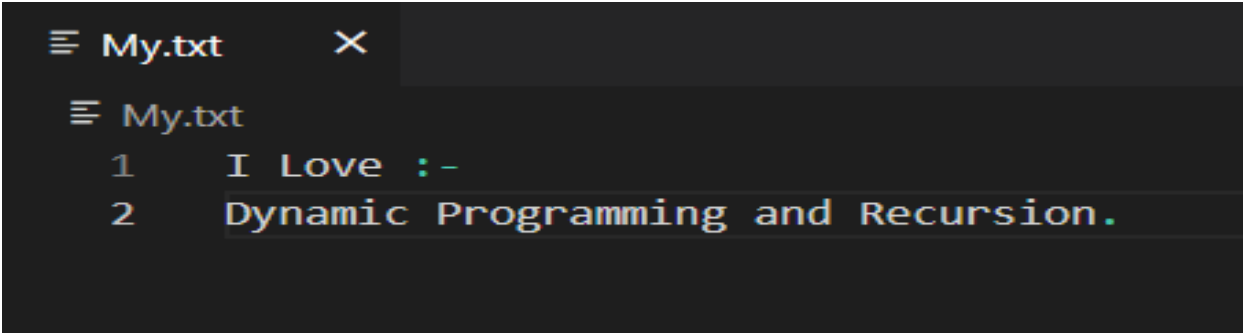
See sometimes, you have connected a pen drive or memory card to your laptop and you have kept one of the files open and you try to eject the card. Then you will get a message that "some program is using your memory card or pen drive". So, you cannot eject it. It means the operating system knows that some program is using some file so it will not allow you to eject. **It means that files are being occupied or that resource is in use.** So, when the program is not used then it should be released. If the program ends, then

automatically the resource will be released. So, it is a good practice to close the resource when you have finished.



```
1 #include<iostream>
2 #include<fstream>
3 using namespace std;
4 int main(){
5     ofstream outfile("My.txt", ios::trunc);
6     outfile<<"I Love :- "<<endl;
7     outfile<<"Dynamic Programming and Recursion."<<endl;
8     outfile.close();
9     return 0;
10 }
```

Output:-



```
My.txt
1 I Love :-
2 Dynamic Programming and Recursion.
```

Now, if you don't want to remove the old data instead you want to append the new data below the old data, then you need to use the `ios::app`

```
1 #include<iostream>
2 #include<fstream>
3 using namespace std;
4 int main(){
5     ofstream outfile("My.txt", ios::app);
6     outfile<<"And Competitive Programming As well"<<endl;
7     outfile.close();
8     return 0;
9 }
```

Output:-

```
≡ My.txt  X
≡ My.txt
1 I Love :-
2 Dynamic Programming and Recursion.
3 And Competitive Programming As well
4
```

How to Open a File in C++?

```
int main() {
    ifstream infile;
    infile.open("my.txt");
    if (!infile)
        cout << "File Cannot be opened";
}
```

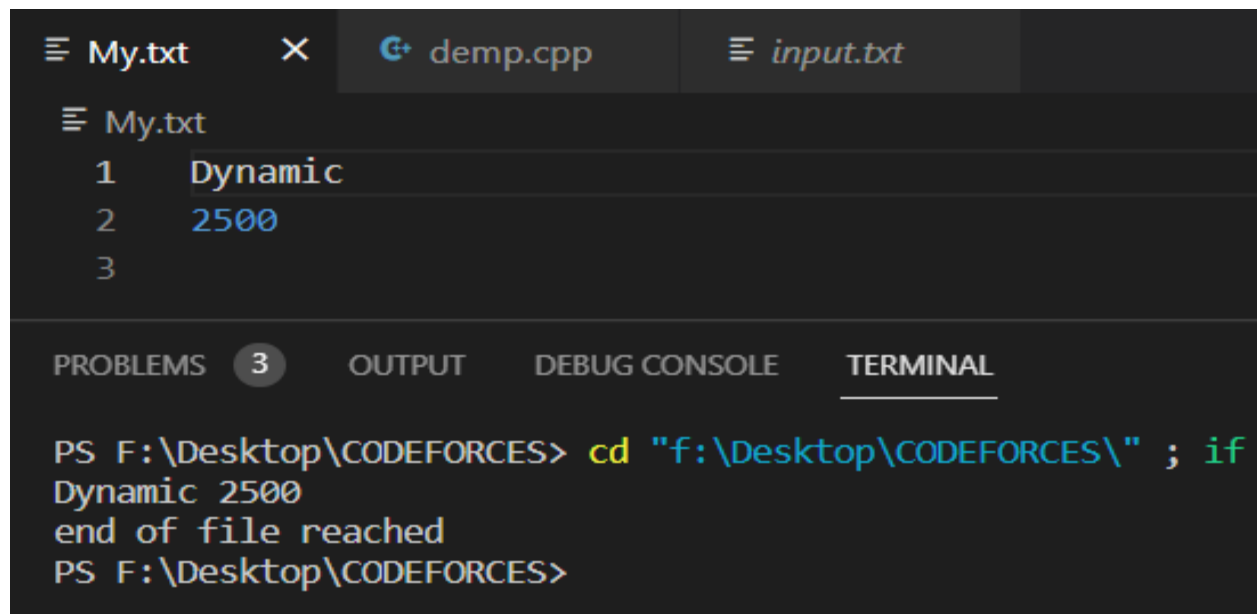
So, the `infile.open("my.txt")` statement will open the file. Now one important thing, when you are reading from a file then the file must be existing. It will not create a new file so the file must be existing at the location where the program is saved.

Next, we have checked for the file status. `if(!infile)` statement will check for whether the file is open or not.

There is another method for checking the file status.

`infile.is_open()`, this method will return true if the file is opened otherwise it will return false.

```
1  #include <iostream>
2  #include <fstream>
3  using namespace std;
4  int main(){
5      ifstream infile;
6      infile.open("my.txt");
7      string str;
8      int x;
9      infile>>str;
10     infile>>x;
11     infile>>x;
12     cout<<str<<" "<<x<<endl;
13     if (infile.eof()){
14         cout<<"end of file reached";
15     }
16     infile.close();
17 }
```



```
My.txt x demp.cpp input.txt
My.txt
1 Dynamic
2 2500
3

PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL
PS F:\Desktop\CODEFORCES> cd "f:\Desktop\CODEFORCES\" ; if
Dynamic 2500
end of file reached
PS F:\Desktop\CODEFORCES>
```

What is eof function in C++?

One important thing, once we have finished reading the content, we have reached the end of the file (eof). So sometimes we need to check whether we have reached the end or not. `infile.eof()`, this statement will return true if we are at the end of the file. So, we have written the print statement "end of file reached".

Example to Understand File Reading in C++:

```

1  #include<iostream>
2  #include<fstream>
3  using namespace std;
4  int main(){
5      ifstream ifs;
6      ifs.open ("my.txt");
7      if (ifs.is_open())
8          cout<<"File is Opened"<<endl;
9      string name;
10     int roll;
11     string branch;
12     ifs >> name >> roll >> branch;
13     ifs.close();
14     cout<<"Name: "<<name<<endl;
15     cout<<"Roll: "<<roll<<endl;
16     cout<<"Branch: "<<branch<<endl;
17     if(ifs.eof())
18         cout<<"End of File Reached";
19 }

```

My.txt x demp.cpp input.txt

My.txt

```

1  Aman_Babu
2  2019017
3  CSE
4

```

PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL

```

PS F:\Desktop\CODEFORCES> cd "f:\Desktop\CODEFORCES\" ; if ($?) { g++ demp.cpp
File is Opened
Name: Aman_Babu
Roll: 2019017
Branch: CSE
End of File Reached
PS F:\Desktop\CODEFORCES>

```

Serialization in C++ :- Serialization is the process of storing and retrieving the state of an object. Suppose we want to store the student information in a file and again we want to retrieve the information of a student from a file. While working with Serialization the class must have a default constructor.

```
1  #include<iostream>
2  #include<fstream>
3  using namespace std;
4  class Student {
5  public:
6      string name;
7      int rollno;
8      string branch;
9      Student(){}
10     Student(string s, int roll_no, string t){
11         name = s;
12         rollno = roll_no;
13         branch = t;
14     }
15     friend ofstream & operator << (ofstream & ofs, Student s);
16     friend ifstream & operator >> (ifstream & ifs, Student & s);
17 };
18 int main(){
19     Student s1("Aman Babu", 2019017, "CSE");
20     ofstream ofs("Student.txt", ios::trunc);
21     ofs<<s1.name<<endl;
22     ofs<<s1.rollno<<endl;
23     ofs<<s1.branch<<endl;
24 }
```

≡ Student.txt X

≡ Student.txt

```
1  Aman Babu
2  2019017
3  CSE
4
```

Why we have stored all the values individually? If we can store the complete object with name, rollno and branch then we can say that we have stored the object. So, **if we can store the whole object at once then we can store an object as well as we can retrieve the object.** Yes. This is possible. We have to overload the operator of the ofs stream in the student class

```
1  #include<iostream>
2  #include<fstream>
3  using namespace std;
4  class Student{
5  public:
6      string name;
7      int rollno;
8      string branch;
9      Student(){}
10     Student(string n, int r, string b){
11         name = n;
12         rollno = r;
13         branch = b;
14     }
15     friend ofstream & operator<<(ofstream &ofs, Student s);
16     friend ifstream & operator>>(ifstream &ifis, Student &s);
17 };
18 ofstream & operator<<(ofstream & ofs, Student s){
19     ofs<<s.name<<endl;
20     ofs<<s.rollno<<endl;
21     ofs<<s.branch<<endl;
22     return ofs;
23 }
24 int main(){
25     Student s1("Aman Babu", 2019017, "CSE");
26     ofstream ofs("Student.txt", ios::trunc);
27     ofs<<s1;
28     ofs.close ();
29 }
```

≡ Student.txt ✕

≡ Student.txt

1 Aman Babu

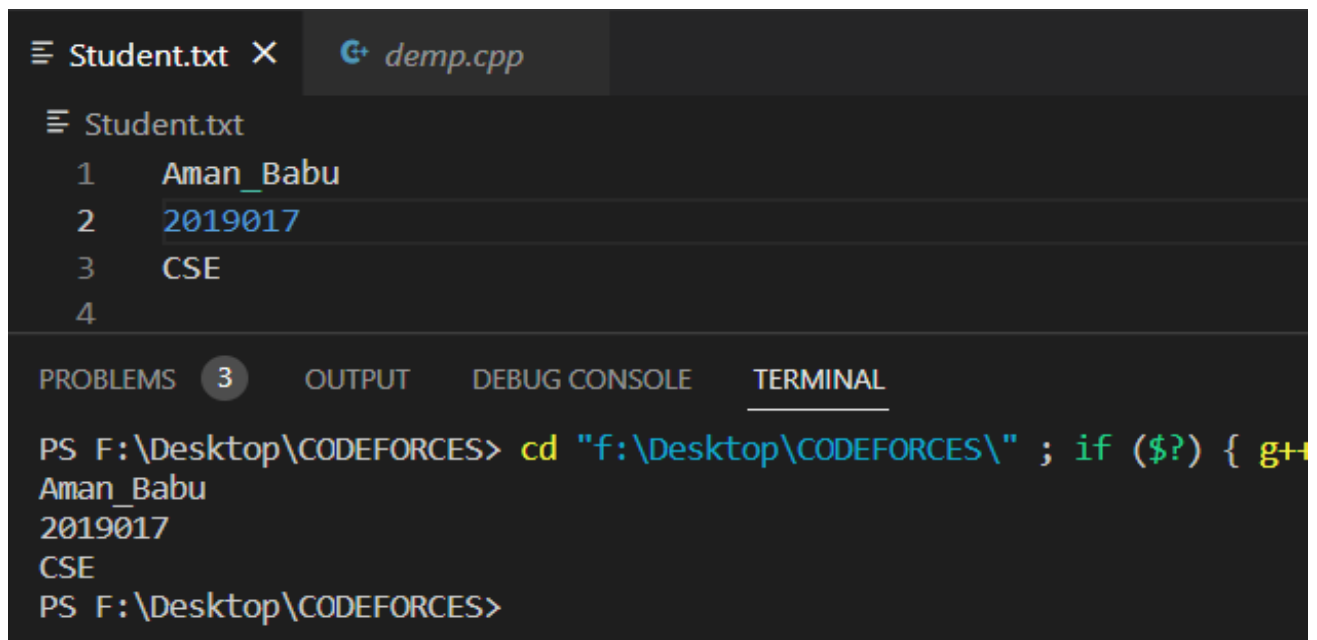
2 2019017

3 CSE

4

Retrieving the State of an Object in C++:-

```
1  #include<iostream>
2  #include<fstream>
3  using namespace std;
4  class Student{
5  public:
6      string name;
7      int rollno;
8      string branch;
9      Student(){}
10     Student(string n, int r, string b){
11         name = n;
12         rollno = r;
13         branch = b;
14     }
15     friend ofstream & operator<<(ofstream &ofs, Student s);
16     friend ifstream & operator>>(ifstream &if, Student &s);
17 };
18 ofstream & operator<<(ofstream & ofs, Student s){
19     ofs<<s.name<<endl;
20     ofs<<s.rollno<<endl;
21     ofs<<s.branch<<endl;
22     return ofs;
23 }
24 ifstream & operator >> (ifstream & ifs, Student & s){
25     ifs>>s.name>>s.rollno>>s.branch;
26     return ifs;
27 }
28 int main(){
29     Student s1("Aman_Babu", 2019017, "CSE");
30     ofstream ofs("Student.txt", ios::trunc);
31     ofs<<s1;
32     ofs.close ();
33     Student s2;
34     ifstream ifs("Student.txt");
35     ifs>>s2;
36     cout<<s2.name<<endl<<s2.rollno<<endl<<s2.branch<<endl;
37     ifs.close();
38 }
```



```
Student.txt X demp.cpp
Student.txt
1 Aman_Babu
2 2019017
3 CSE
4
PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL
PS F:\Desktop\CODEFORCES> cd "f:\Desktop\CODEFORCES\" ; if ($?) { g++
Aman_Babu
2019017
CSE
PS F:\Desktop\CODEFORCES>
```

Text and Binary Files in C++:

There are two types of files:

- Text Files
- Binary Files

Difference between Text and Binary Files in C++ with Examples:

Text files are human-readable whereas binary files are machine-readable.

Suppose we write a number that is 13. Then how is it written in the text file and binary file? Let's see the difference.

13 is an integer value. The binary form of 13 is 1101. So how many bytes do integers take? In most compilers, an integer takes 4 bytes but to make our explanation easy, we consider an integer takes 2 bytes. So, suppose 13 (integer) is taking 2 bytes then how many bits are there in 1101? **0000 0000 0000 1101**

Total 16 bits of a binary number. So, the same binary form is stored in the binary file with all 16 bits. That's why we called this a binary file.

Then what are text files? 13 will not be written in the text file. it will convert into ASCII. **The ASCII code of 1 is 49 and for 3 the ASCII code is 51.** Then what is the binary form of 49 and 51?

49 - 110001, 51 - 110011

These are the binary forms of 49 and 51. ASCII codes take 8 bits of binary so, **49 - 00110001, 51 - 00110011**

0011 0001 0011 0011 (This will be stored in the text file that is)

(ASCII code of 1) + (ASCII code of 3) = 49 + 51 =

(binary code of 49) + (binary code of 51) = 0011000100110011

Both text and binary files stored the binary number. Then how text files are human-readable?

Suppose we have a text file that has stored 13. When we open this file in notepad then what will notepad do? For every 8 bits, it will be converted into ASCII and then display that symbol.

0011 0001 0011 0011

So, for this binary number, the first 8 bits will be converted into ASCII which is 1 and the next 8 bits will be converted into ASCII which is 3. So, 1 and 3 will be displayed in a notepad.

And what about binary files? Suppose we have a binary file that contains the following code.

0000 0000 0000 1101

In a binary file, the first 8 bits will be taken. **In this case, all the bits are 0, and 0 is the ASCII code of some unknown garbage symbol. So, we will get some junk characters.** If you open this file in notepad then it might not show any meaningful symbols because the first 8 bits are not making any meaningful ASCII code. That's it. We cannot read and understand it because that is a pure binary form or we can say it is machine-understandable.

If you are reading from the text file then you can use the insertion and extraction operator for reading and writing the data in the form of text.

And if you want to read and write it in the form of binary then the first thing in C++ that you have to use is **iso::binary mode**. And also there are functions available for reading and writing that are **read()** and **write()**.

read() is available in the file input stream and **write()** is available in the file output stream.

Which file is faster? Binary file or text file?

The binary file is faster than the text file. Because text file needs conversion (symbols to ASCII and ASCII to binary). But in the binary file, there is no conversion required.

Which file takes more space?

Text files will take more space and binary will take less space.

Suppose we have the 4-digit number so a text file will take 4 bytes but a binary file will take 2 bytes.

Manipulators in C++

Manipulators are helping functions in C++ that are used to modify the input/output stream. What it means, **it will not modify the value of a variable, it will only modify the streams or format of streams using the insertion (<<) and extraction (>>) operators.**

- Manipulators are special functions that can be included in the I/O statement to alter the format parameters of a stream.
- Manipulators are operators that are used to format the data display.
- To access manipulators, the file `iomanip` should be included in the program.

Manipulators are used for enhancing streams or formatting streams. For writing the data, we can adopt some formats. For example, a common manipulator that we used is the `endl` that is used for the newline. Instead of `endl`, we can also say that `cout << "\n";`

This will also print a new line. So, `endl` is a manipulator which is used for formatting stream. So, it is useful for formatting output streams.

Integer Manipulators in C++:-

hex - It will display the data in hexadecimal.

oct - It will display data in the octal form.

dec - To display data in decimal form.

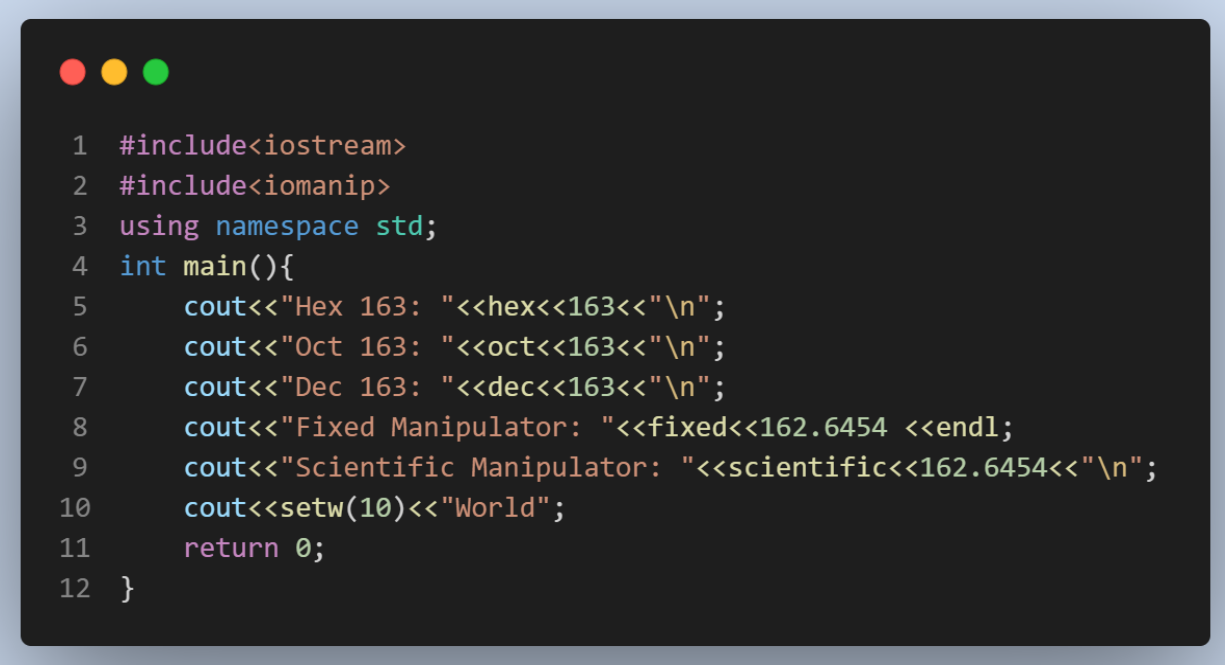
Float Manipulators in C++:-

fixed: It will show in the fixed floating-point number.

scientific: It will display the number in scientific form or exponent form.

There are other manipulators also available.

setw - It will set some amount of space for displaying the data. For example,



```
1 #include<iostream>
2 #include<iomanip>
3 using namespace std;
4 int main(){
5     cout<<"Hex 163: "<<hex<<163<<"\n";
6     cout<<"Oct 163: "<<oct<<163<<"\n";
7     cout<<"Dec 163: "<<dec<<163<<"\n";
8     cout<<"Fixed Manipulator: "<<fixed<<162.6454 <<endl;
9     cout<<"Scientific Manipulator: "<<scientific<<162.6454<<"\n";
10    cout<<setw(10)<<"World";
11    return 0;
12 }
```

Output:-

```
PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL
PS F:\Desktop\CODEFORCES> cd "f:\Desktop\CODEFORCES\" ; if ($?) {
Hex 163: a3
Oct 163: 243
Dec 163: 163
Fixed Manipulator: 162.645400
Scientific Manipulator: 1.626454e+002
World
```

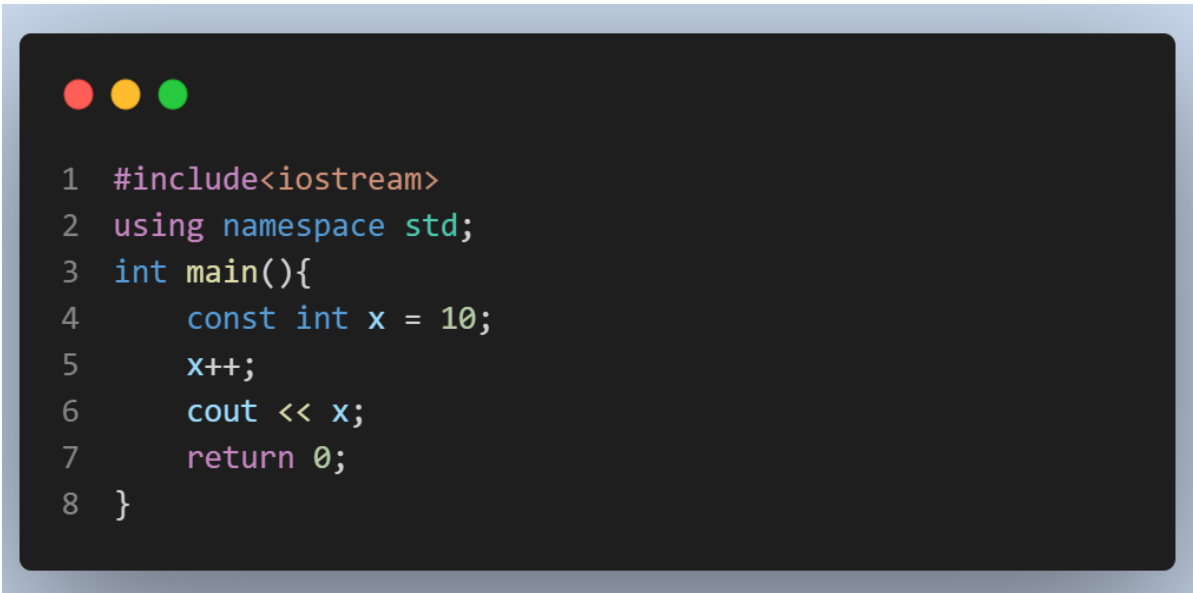
Constants in C++

As the name suggests, Constants in C++ are given to such variables or values which cannot be modified or changed once they are defined. That means they are fixed values in a program, unlike the variables whose value can be altered.

There are two ways to define constants in C++.

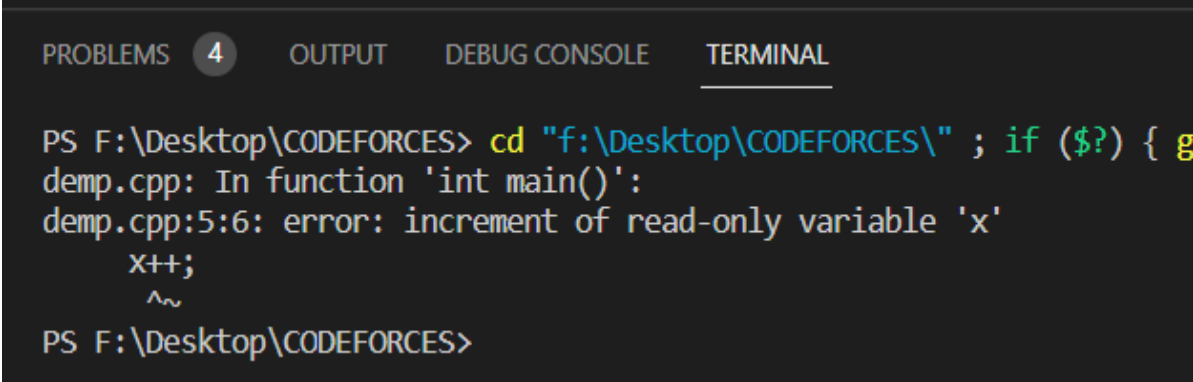
- By using the const keyword
- By using #define preprocessor

```
int main(){
    const int x = 10;
    ✖ x++;
    cout << x;
}
```



```
1 #include<iostream>
2 using namespace std;
3 int main(){
4     const int x = 10;
5     x++;
6     cout << x;
7     return 0;
8 }
```

Output:-



```
PROBLEMS 4 OUTPUT DEBUG CONSOLE TERMINAL
PS F:\Desktop\CODEFORCES> cd "f:\Desktop\CODEFORCES\" ; if ($?) { g
demp.cpp: In function 'int main()':
demp.cpp:5:6: error: increment of read-only variable 'x'
    x++;
    ^~
PS F:\Desktop\CODEFORCES>
```

These variables are also called read-only variables.

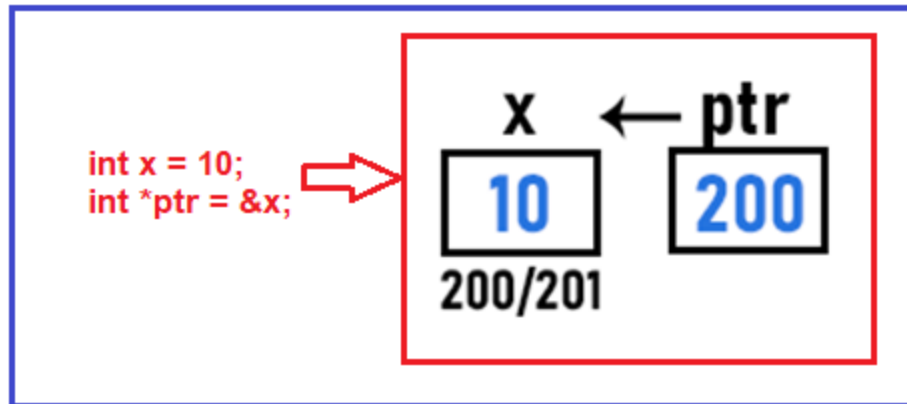
#define x 10

What is the difference between **#define** and **const** in C++?

- **#define** is a preprocessor directive and it is performed before the compilation process starts. But **const** is an identifier that will consume memory that cannot be modified.
- **#define** is just a symbolic constant and **const** is a constant identifier.
- **#define** will not consume memory, but **const** will consume memory according to the data type.
- **#define** is not a part of the language, it is a pre-compiler whereas **const** is a part of the language that is part of the compiler.

Constant Pointers in C++:-

```
int main()
{
    int x = 10;
    int *ptr = &x;
    ++*ptr;
    cout << x;
}
```



Then we have written `++*ptr`. So, `*ptr` will increment the value of `x` from 10 to 11 as follows.



But

```
int main(){  
    int x = 10;  
    const int *ptr = &x;  
    ++*ptr;  
    cout << x;  
}
```

In this case, the pointer `ptr` can point to `x` and **access or read** `x`, but it **cannot modify the value** `x`. **So here, `++*ptr` is not allowed**. So, by using the constant pointer, we cannot modify the data.

This can be written in other ways also, like,

int const *ptr = &x; OR const int *ptr = *x;

Suppose we have one more variable y and it is having a value of 30 (int y = 30;). So, can we point ptr at y? **Yes. By writing, ptr = &y,** ptr will point at y. So, we can make the pointer point to some other data, but you cannot modify that. Here also, we cannot modify the value of y.

```
1  #include<iostream>
2  using namespace std;
3  int main(){
4      int x = 10;
5      int y = 30;
6      const int *ptr = &x;
7      ptr = &y; //Allowed
8      //++*ptr; //Not Allowed
9      cout << *ptr;
10 }
11 //output
12 30
```

Another Usage of Constants in C++:

- int * const ptr = &x;

Now ptr is a constant pointer to an integer. So, who is the constant here? Pointer ptr is constant here. What does it mean? It means once the pointer ptr is pointing at x, it cannot be

changed. After that, you will be unable to change it to another variable. **We cannot write `ptr = &y`** because `ptr` is constant. So, the address in that pointer cannot be changed.

```
1  #include<iostream>
2  using namespace std;
3  int main(){
4      int x = 10;
5      int y = 30;
6      int * const ptr = &x;
7      ++*ptr; // Allowed
8      ptr = &y; //Not Allowed
9      cout<<*ptr;
10 }
```

Output :-

```
PROBLEMS 4 OUTPUT DEBUG CONSOLE TERMINAL
PS F:\Desktop\CODEFORCES> cd "f:\Desktop\CODEFORCES\" ; if ($?) { g++ demp.cpp; }
demp.cpp: In function 'int main()':
demp.cpp:8:12: error: assignment of read-only variable 'ptr'
    ptr = &y; //Not Allowed
       ^
PS F:\Desktop\CODEFORCES>
```

Two constants in C++:-

- `const int * const ptr = &x;`

Two constants in C++:-

✗ `ptr = &y;`
✗ `++(*ptr);`

We cannot modify the data by writing `++ *ptr` and we cannot assign the pointer to another data like `ptr = &y`.



```
1 #include<iostream>
2 using namespace std;
3 int main(){
4     int x = 10;
5     int y = 30;
6     const int * const ptr = &x;
7     ++*ptr; //const int * const ptr = &x; Allowed
8     ptr = &y; //Not Allowed
9     cout<<*ptr;
10    return 0;
11 }
```

Output :-

```
PROBLEMS 5 OUTPUT DEBUG CONSOLE TERMINAL
PS F:\Desktop\CODEFORCES> cd "f:\Desktop\CODEFORCES\" ; if ($?) { g++ demp.cpp
demp.cpp: In function 'int main()':
demp.cpp:7:8: error: increment of read-only location '*(const int*)ptr'
    ++*ptr; //const int * const ptr = &x; Allowed
    ^~~~
demp.cpp:8:12: error: assignment of read-only variable 'ptr'
    ptr = &y; //Not Allowed
    ^
PS F:\Desktop\CODEFORCES>
```

There are 3 types of constant pointers in C++.

- **The pointer can be constant:** Pointer cannot be modified i.e. `ptr = &y` is not valid here.

- **Data can be constant:** Pointer cannot modify the data i.e. `++*ptr` is not valid.
- **Both the pointer and data can be constant:** Here pointer cannot be modified i.e. `ptr = &y` is not valid, and data is also can't be modify i.e. `++*ptr` is not valid.

Const Keyword in C++ Functions:-

```
1  #include<iostream>
2  using namespace std;
3  class Demo{
4  public:
5      int x = 10;
6      int y = 20;
7      void Display() const{
8          x++; //Not Allowed
9          cout << x << " " << y << endl;
10     }
11 };
12 int main(){
13     Demo d;
14     d.Display();
15 }
```

Output:-

```
PROBLEMS 4 OUTPUT DEBUG CONSOLE TERMINAL
PS F:\Desktop\CODEFORCES> cd "f:\Desktop\CODEFORCES\" ; if ($?) { g++ demp.cpp -o demp.cpp: In member function 'void Demo::Display() const':
demp.cpp:8:10: error: increment of member 'Demo::x' in read-only object
    x++; //Not Allowed
    ^~
PS F:\Desktop\CODEFORCES>
```

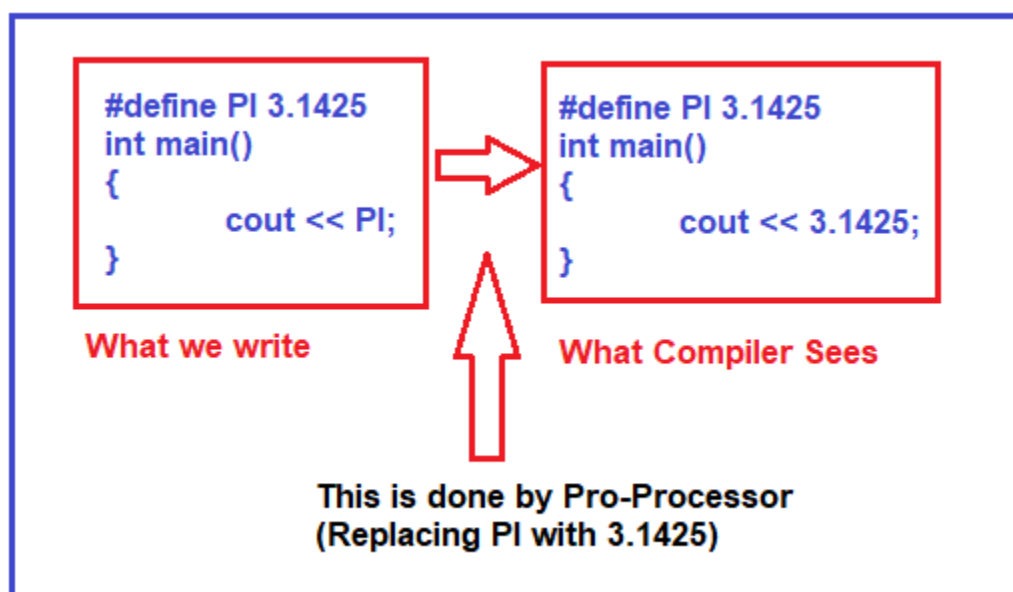
Another use of const keyword:-

```
void fun(const int &x, const int &y){  
    ---  
}
```

So, parameters can also be made constant. Now a and b cannot be changed through function fun. We cannot write x++ or y++ here.

Preprocessor Directives in C++

Macros or Preprocessor Directives in C++ are instructions to the compiler. **We can give some instructions to the compiler so that before the compiler starts compiling a program, it can follow those instructions and perform those instructions.** The most well-known Macros or Preprocessor Directives that we used in our program is **#define**.

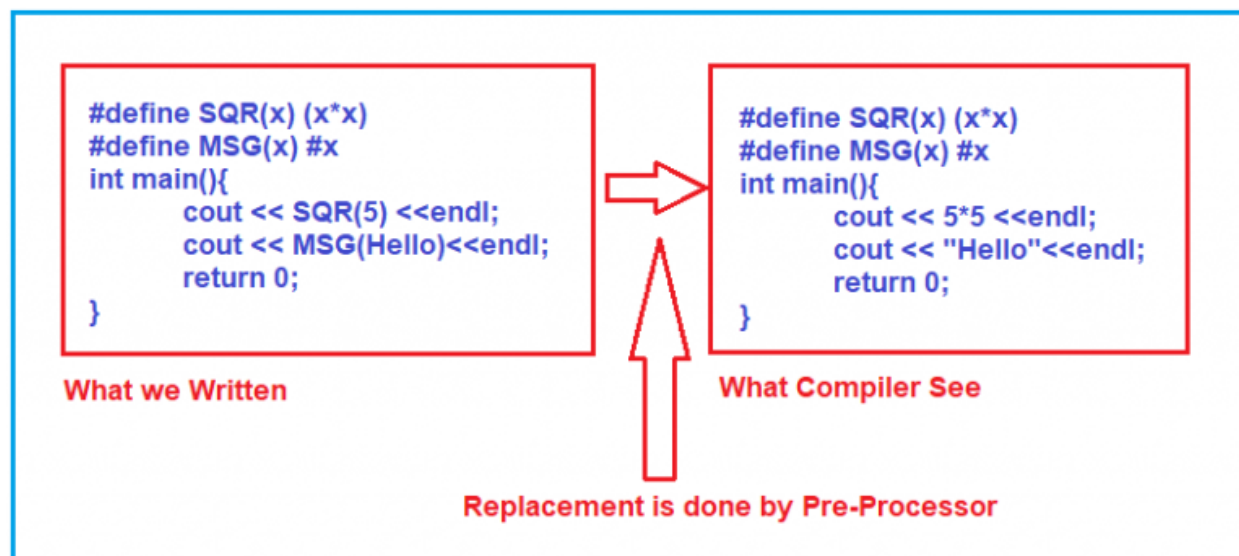


#define c cout

Now, can we write `c << 10`? Yes. What will happen? This `c` will be replaced by `cout` before the compilation.

Define Function using #define Preprocessor Directive in C++:-

```
#define SQR(x) (x*x)
#define MSG(x) #x
int main(){
    cout << SQR(5) << endl;
    cout << MSG(Hello) << endl;
    return 0;
}
```



With `#x`, we created another function, `MSG(x)`. Whatever the parameters we send in `MSG`, that will be converted into a string.

So, if we write `cout << MSG(Hello)`, then it means "Hello". So, we have given Hello without double-quotes. But `MSG (Hello)` will be replaced by "Hello".

`#ifndef` Directive in C++, `#ifndef`. It means if not defined.

```
#ifndef
    #define PI 3.1425
#endif
```

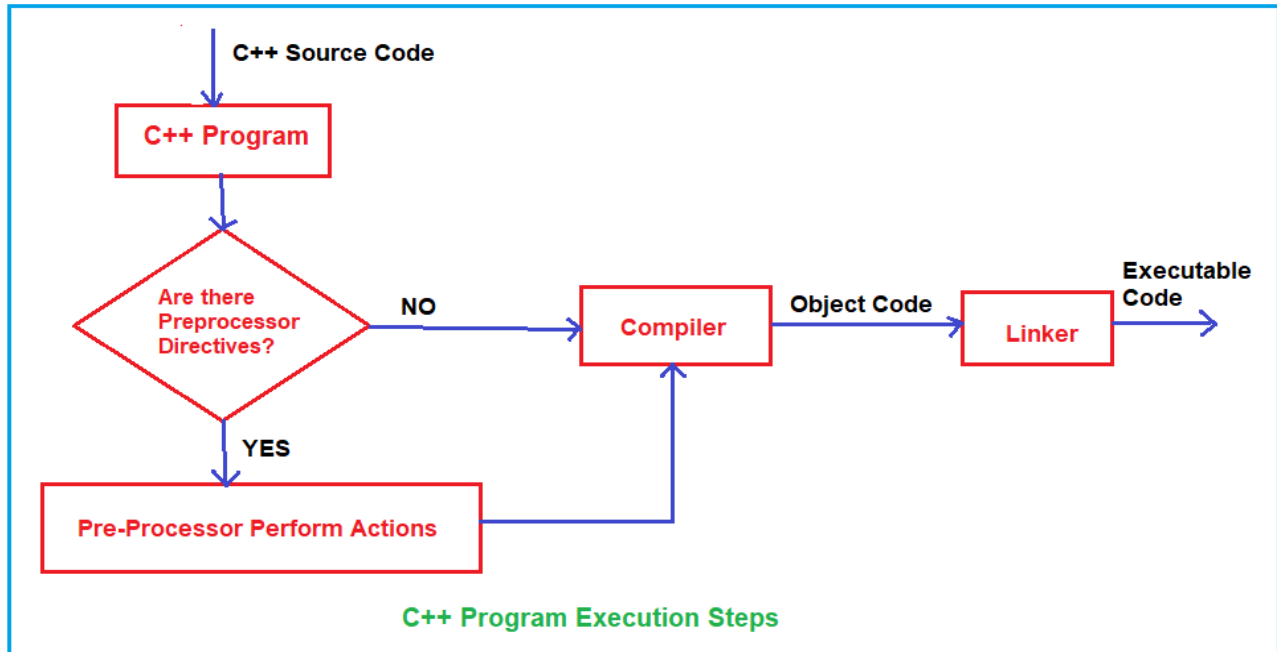
This PI is defined if it is not already defined, then only it will be defined, otherwise, it will not be defined again.

```
1  #include <iostream>
2  using namespace std;
3  #define max(x, y) (x > y ? x : y)
4  #ifndef PI
5      #define PI 3.1425
6  #endif
7  int main(){
8      cout<<PI<<endl;
9      cout<<max(121, 125)<<endl;
10     return 0;
11 }
```

PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL

```
PS F:\Desktop\CODEFORCES> cd "f:\Desktop\CODEFORCES\" ; if ($?) { g++ d
3.1425
125
PS F:\Desktop\CODEFORCES>
```

How a C++ Program Executes?



Namespaces in C++ :- Namespaces are used for removing name conflicts in C++. If you are writing multiple functions with the same name but are not overloaded, they are independent functions. They are not a part of the class.

```
void fun (){\n    cout << "First";\n}\nvoid fun(){\n    cout << "Second";\n}\nint main (){\n    fun();\n}
```

So, we have two functions of name fun and one main function. The main function is to call the function fun. So, which fun function will be called? First fun or second fun? **First of all, the compiler will not compile our program. The compiler will say that we are redefining the same function multiple times.**

But we want the same function but we want to remove this ambiguity. We have to remove the name conflicts. So, for this, we can use namespaces in C++. Let us define our namespace as follows:

```
namespace First{  
    void fun (){  
        cout << "First";  
    }  
}  
  
namespace Second{  
    void fun(){  
        cout << "Second";  
    }  
}
```

```
First::fun();  
Second::fun();
```

So first we have to write the namespace, then the scope resolution operator, and then the function name.

```

1  #include <iostream>
2  using namespace std;
3  namespace First{
4      void fun(){
5          cout<<"First"<<endl;
6      }
7  }
8  namespace Second{
9      void fun(){
10         cout<<"Second"<<endl;
11     }
12 }
13 int main(){
14     First::fun ();
15     Second::fun ();
16     return 0;
17 }

```

Output :-

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

PS F:\Desktop\CODEFORCES> cd "f:\Desktop\CODEFORCES\" ; if ($?) { g
First
Second
PS F:\Desktop\CODEFORCES>

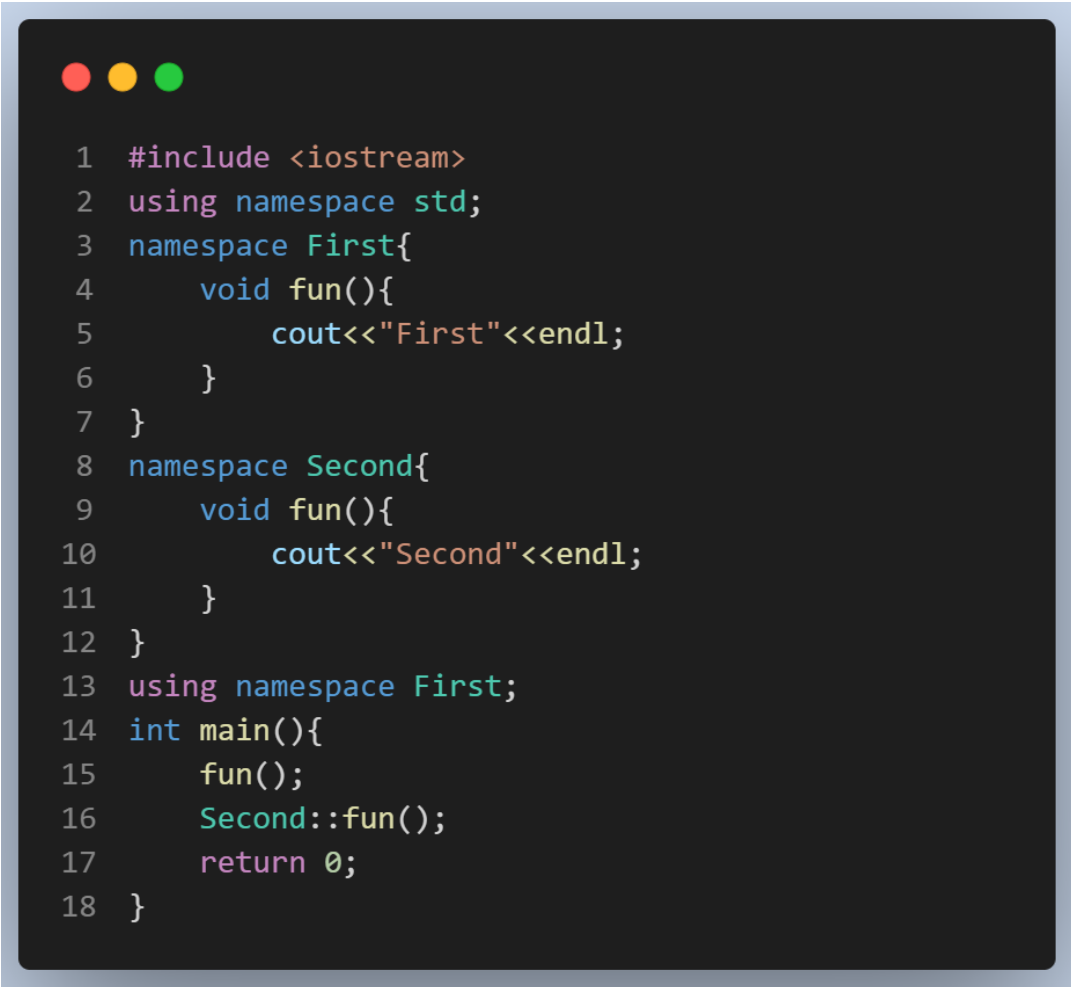
```

A namespace is a container that holds a set of identifiers. **These identifiers can be variables, functions, or classes**, and they are used to organize and structure code. A namespace helps to prevent naming conflicts by providing a way to group related identifiers together and give them a unique name.

using namespace First;

Now when we call the function fun anywhere in the program, it will call fun inside the first namespace. If you still want to call the second namespace function, then you can write,

Second::fun();



```
1  #include <iostream>
2  using namespace std;
3  namespace First{
4      void fun(){
5          cout<<"First"<<endl;
6      }
7  }
8  namespace Second{
9      void fun(){
10         cout<<"Second"<<endl;
11     }
12 }
13 using namespace First;
14 int main(){
15     fun();
16     Second::fun();
17     return 0;
18 }
```

Output:-

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

PS F:\Desktop\CODEFORCES> cd "f:\Desktop\CODEFORCES\" ; if ($?) { g
First
Second
PS F:\Desktop\CODEFORCES>
```

In our C++ program, we have been using namespace std, so there is one namespace std that has the cin and cout objects. So that's why we just write a single line using namespace std; and we can use cin and cout objects. Otherwise, we have to write like this,

`std::cout << "Hello";`

Destructors in C++ :-

A destructor in C++ is also a member function like a constructor which is also invoked automatically when the object goes out of scope or we can also explicitly call the destructor to destroy the object by making a call to delete.

A destructor has the same name as the class name as a constructor, but to differentiate between them the destructor function is preceded by a tilde (~) symbol

```
class Test{
    public:
        Test(){
            cout << "Test Object Created" << endl;
        }
        ~Test(){
            cout << "Test Object Destroyed" << endl;
        }
};
```

When the Destructor function will be Called in C++?

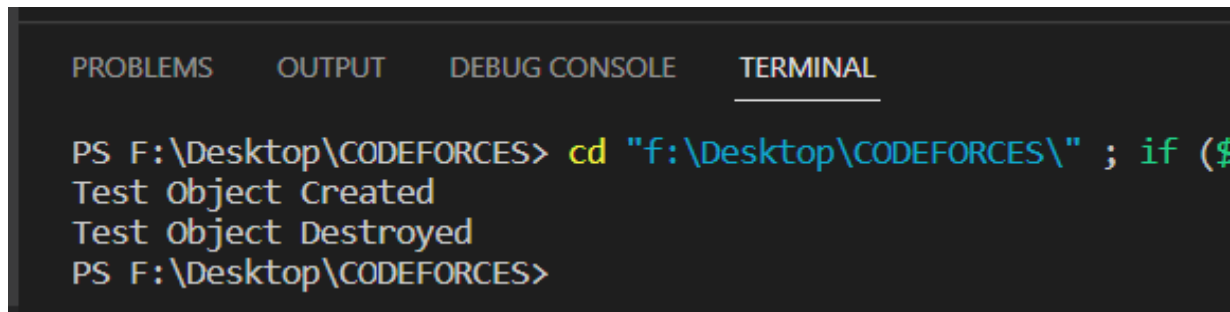
This destructor function will be called when the object will be destroyed. We know once the main function ends, automatically all the objects that are associated with the program get destroyed. **So, the constructor is called when the object is created and the destructor is called when the object is destroyed.**

```
Test *obj = new Test();  
delete obj;
```

As you can see in the above example, we have written delete obj. So, when we are creating an object by writing a **new Test()**, the constructor will be called. And when we delete that object by writing delete obj then destructor will be called.

```
1  #include <iostream>  
2  using namespace std;  
3  class Test{  
4  public:  
5      Test(){  
6          cout<<"Test Object Created" << endl;  
7      }  
8      ~Test(){  
9          cout<<"Test Object Destroyed" << endl;  
10     }  
11 };  
12 int main(){  
13     Test *obj = new Test();  
14     delete obj;  
15     return 0;  
16 }
```

Output:-



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

PS F:\Desktop\CODEFORCES> cd "f:\Desktop\CODEFORCES\" ; if ($?) { g++ test.cpp -std=c++11 -o test.exe ; test.exe }
Test Object Created
Test Object Destroyed
PS F:\Desktop\CODEFORCES>
```

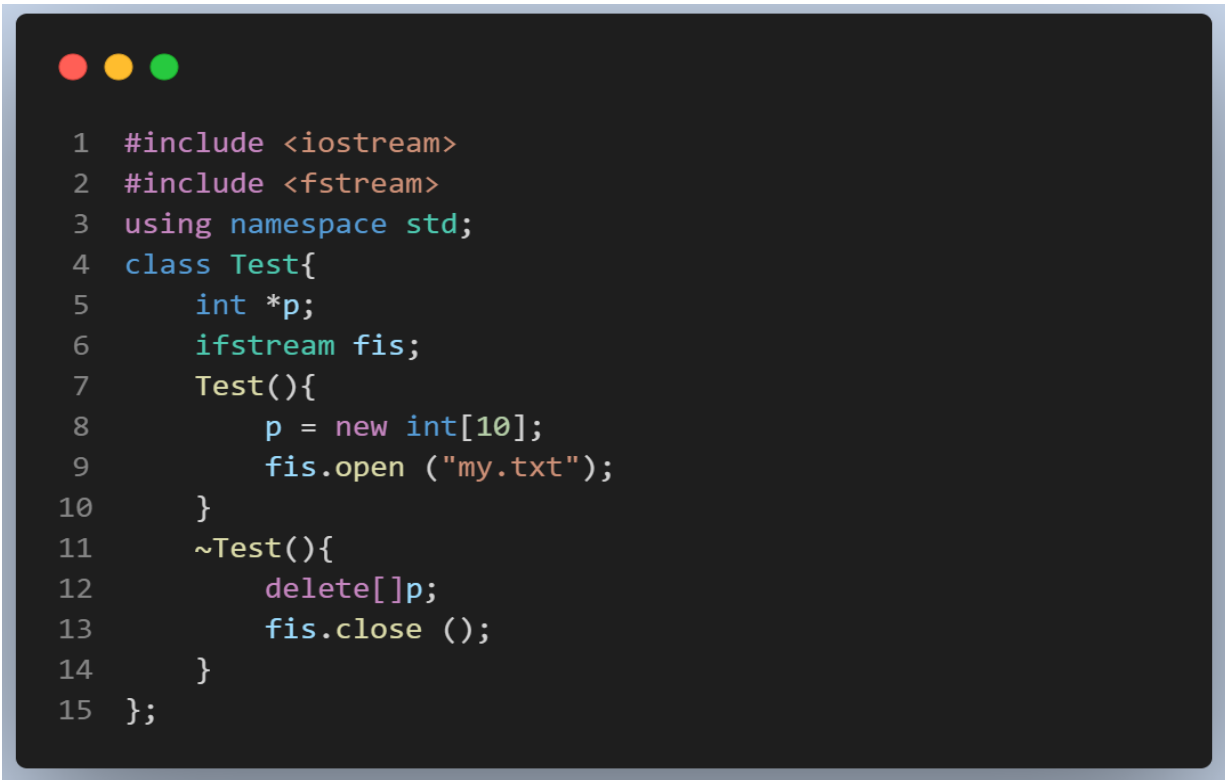
If an object is created dynamically then it should also be deleted when not required. So, when we say `delete obj`, the destructor function will be called.

Is it Mandatory to Define a Destructor in C++?

No, it is not mandatory to define a destructor in C++. If as a programmer we do not define a destructor, the compiler will provide a default one automatically. And for many classes, this compiler-provided default destructor is sufficient. **We only need to define a destructor explicitly in our class when the class handles external resources that need to be released.**

What is the use of Destructor in C++?

The constructor is used for initialization purposes. It is also used for allocating resources. Similarly a destructor is used for deallocating resources or releasing resources. we have to deallocate the resources. Which type of resources? When you are acquiring any external things like heap memory, file, network connection, etc. these are all external sources.



```
1  #include <iostream>
2  #include <fstream>
3  using namespace std;
4  class Test{
5      int *p;
6      ifstream fis;
7      Test(){
8          p = new int[10];
9          fis.open ("my.txt");
10     }
11     ~Test(){
12         delete[]p;
13         fis.close ();
14     }
15 };
```

Here we have a class called Test. Inside this Test class, we have a pointer p. We have allocated this p dynamically inside the constructor. Then inside the destructor, we must delete the allocated memory otherwise this will cause memory leak problems.

Can we overload the constructor? Can we have multiple constructors? Yes. Can we have multiple destructors? No. Can the constructor or destructor return anything? No. All the rules of constructors are followed on destructor except destructor cannot be overloaded. A destructor can be virtual also.

Example to Understand Static Allocation of an object in C++

```
1  #include <iostream>
2  using namespace std;
3  class Test{
4  public:
5      Test(){
6          cout<<"Test Created"<<endl;
7      }
8      ~Test(){
9          cout<<"Test Destroyed"<<endl;
10     }
11 };
12 void fun(){
13     Test t;
14 }
15 int main(){
16     fun();
17 }
```

Output:-

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS F:\Desktop\CODEFORCES> cd "f:\Desktop\CODEFORCES\" ; if (
Test Created
Test Destroyed
PS F:\Desktop\CODEFORCES>
```

Example to Understand Dynamic Allocation of Object in C++

```
1  #include <iostream>
2  using namespace std;
3  class Test{
4  private:
5      int *p;
6  public:
7      Test(){
8          p = new int[10];
9          cout<<"Test Created"<<endl;
10     }
11     ~Test(){
12         delete[]p;
13         cout<<"Test Destroyed"<<endl;
14     }
15 };
16 void fun(){
17     Test *t = new Test ();
18     delete t;
19 }
20 int main(){
21     fun();
22 }
23
```

Output:-

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

PS F:\Desktop\CODEFORCES> cd "f:\Desktop\CODEFORCES\" ; if ($?) {
Test Created
Test Destroyed
PS F:\Desktop\CODEFORCES> 
```

Note :- If we are not writing `delete []p` then the destructor will not be called.

Rules to be Followed While Declaring Destructors in C++:-

- C++ Destructors do not accept arguments.
- You cannot return value from a destructor even if a void is not allowed. So, the destructor should not have any return type.
- Destructors in C++ cannot be declared const, volatile, or static. **However, destructors in C++ can be invoked for the destruction of objects which are declared as const, volatile, or static.**
- Destructors in C++ can be declared as virtual.

When Destructor Function Called in C++?

- A local (automatic) object with block scope goes out of scope.
- An object allocated using the new operator is explicitly deallocated using delete.
- The lifetime of a temporary object ends.
- A program ends and global or static objects exist.
- The destructor is explicitly called using the destructor function's fully qualified name.

Virtual Destructors in C++ :-

Example to Understand Destructors inheritance in C++:

```
1  #include <iostream>
2  using namespace std;
3  class Base{
4  public:
5      Base(){
6          cout<<"Base Class Constructor"<<endl;
7      }
8      ~Base(){
9          cout<<"Base Class Destructor"<<endl;
10     }
11 };
12 class Derived:public Base{
13 public:
14     Derived(){
15         cout<<"Derived Class Constructor"<<endl;
16     }
17     ~Derived(){
18         cout<<"Derived Class Destructor"<<endl;
19     }
20 };
21 int main(){
22     Derived d;
23     return 0;
24 }
```

Output :-

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

PS F:\Desktop\CODEFORCES> cd "f:\Desktop\CODEFORCES\" ; if ($?) {
Base Class Constructor
Derived Class Constructor
Derived Class Destructor
Base Class Destructor
PS F:\Desktop\CODEFORCES>
```

Here, we have a Base class. It is having two public members functions that are Base() and ~Base() i.e. one public constructor and one public destructor.

Next, we have a Derived class which is inherited from the Base class. It is also having two public member functions that are constructor and destructor.

How are the Destructors Called in C++?

First, the destructor of the Derived class will be called then the destructor of the Base class will be called. The below messages will be printed on the screen.

Derived Class Destructor

Base Class Destructor

So, if you are creating an object of a derived class and when it is being destroyed, first, the destructor of the Derived class will be called then the destructor of the Base class will be called.

So, in contrast to the constructor, destructors are called from the bottom-up approach.

```

1  #include <iostream>
2  using namespace std;
3  class Base{
4  public:
5      Base(){
6          cout<<"Base Class Constructor"<<endl;
7      }
8      ~Base(){
9          cout<<"Base Class Destructor"<<endl;
10     }
11 };
12 class Derived:public Base{
13 public:
14     Derived(){
15         cout<<"Derived Class Constructor"<<endl;
16     }
17     ~Derived(){
18         cout<<"Derived Class Destructor"<<endl;
19     }
20 };
21 int main(){
22     Base *p = new Derived();
23     delete p;
24     return 0;
25 }

```

Output :-

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

PS F:\Desktop\CODEFORCES> cd "f:\Desktop\CODEFORCES\" ; if ($?) {
Base Class Constructor
Derived Class Constructor
Base Class Destructor
PS F:\Desktop\CODEFORCES>

```

Virtual Destructors in C++:

A Virtual Destructor in C++ is **used to release the memory space which is allocated by the derived class object while deleting the object of the derived class using a base class pointer object.**

Here, inside the Main method, we have a Base class pointer and the object is of the Derived class.

Then on the next line, we are deleting p because dynamically created memory should be deleted when not required using the delete operator.

So new Derived object will be destroyed. Which constructor will be called? Remember in C++, the function calls are depending upon the pointer, not upon the object.

So, the pointer p is of the Base class. So only the Base class destructor will be called. The derived class destructor will not be called.

But suppose you want the destructor of the derived class should also be called and then the Base class destructor called. then we have to write virtual keyword before the name of Base class Destructor as,

```
virtual ~Base ()  
{  
    cout << "Base Class Destructor" << endl;  
}
```

If we write virtual keyword there, then even though the pointer is of Base class and object is of Derived class and when the object is deleted then first, **the destructor of Derived class will be called and then the destructor of Base class will be called.**

```
1  #include <iostream>
2  using namespace std;
3  class Base{
4  public:
5      Base(){
6          cout<<"Base Class Constructor"<<endl;
7      }
8      virtual ~Base(){
9          cout<<"Base Class Destructor"<<endl;
10     }
11 };
12 class Derived:public Base{
13 public:
14     Derived(){
15         cout<<"Derived Class Constructor"<<endl;
16     }
17     ~Derived(){
18         cout<<"Derived Class Destructor"<<endl;
19     }
20 };
21 int main(){
22     Base *p = new Derived();
23     delete p;
24     return 0;
25 }
```

Output:-

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

PS F:\Desktop\CODEFORCES> cd "f:\Desktop\CODEFORCES\" ; if ($?) {
Base Class Constructor
Derived Class Constructor
Derived Class Destructor
Base Class Destructor
PS F:\Desktop\CODEFORCES>
```

So, this is how virtual destructors work.

If you don't make the Base class destructor virtual and you are using a pointer of the Base class and object of the Derived class **then only the resources acquired by the Base class code will be released. But the resources acquired by Derived class code will not be released because that will not execute.**

So, this must also execute. If you are using the Base class pointer and Derived class object then you should make the Base class destructor virtual.

Pure Virtual Destructors in C++ :-

The only difference between Virtual Destructor and Pure Virtual Destructor in C++ is that the **pure virtual destructor will make the class Abstract**, hence we cannot create an object of that class but we can create a pointer of that class.

There is no requirement to implement pure virtual destructors in the derived classes as we are going to create instances of the derived class which are going to be pointed by the Base class pointer.

```

1  #include <iostream>
2  using namespace std;
3  class Base{
4  public:
5      Base(){
6          cout<<"Base Class Constructor"<<endl;
7      }
8      virtual ~Base() = 0;
9  };
10 Base::~~Base(){
11     cout<<"Base Class Destructor"<<endl;
12 }
13 class Derived:public Base{
14 public:
15     Derived(){
16         cout<<"Derived Class Constructor"<<endl;
17     }
18     ~Derived(){
19         cout<<"Derived Class Destructor"<<endl;
20     }
21 };
22 int main(){
23     Base *p = new Derived();
24     delete p;
25     return 0;
26 }

```

Output:-

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```

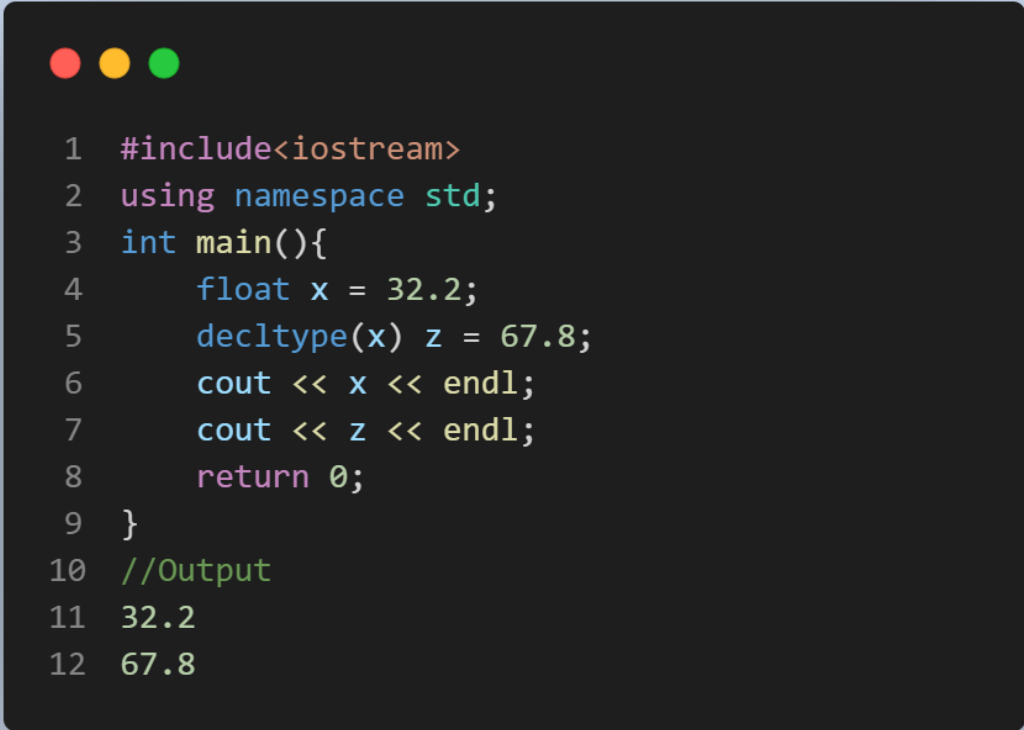
PS F:\Desktop\CODEFORCES> cd "f:\Desktop\CODEFORCES\" ; if ($?) { g++ der
Base Class Constructor
Derived Class Constructor
Derived Class Destructor
Base Class Destructor
PS F:\Desktop\CODEFORCES>

```

Some features of C++ 11

Auto Keyword in C++ :- The auto keyword is a very powerful and useful feature of C++ 11. When a programmer uses the library functions or some functions of the built-in classes then he/she doesn't have to know about the data type. We can simply use the auto declaration to do it automatically. It saves the programmer's time.

Decltype in C++ 11:-



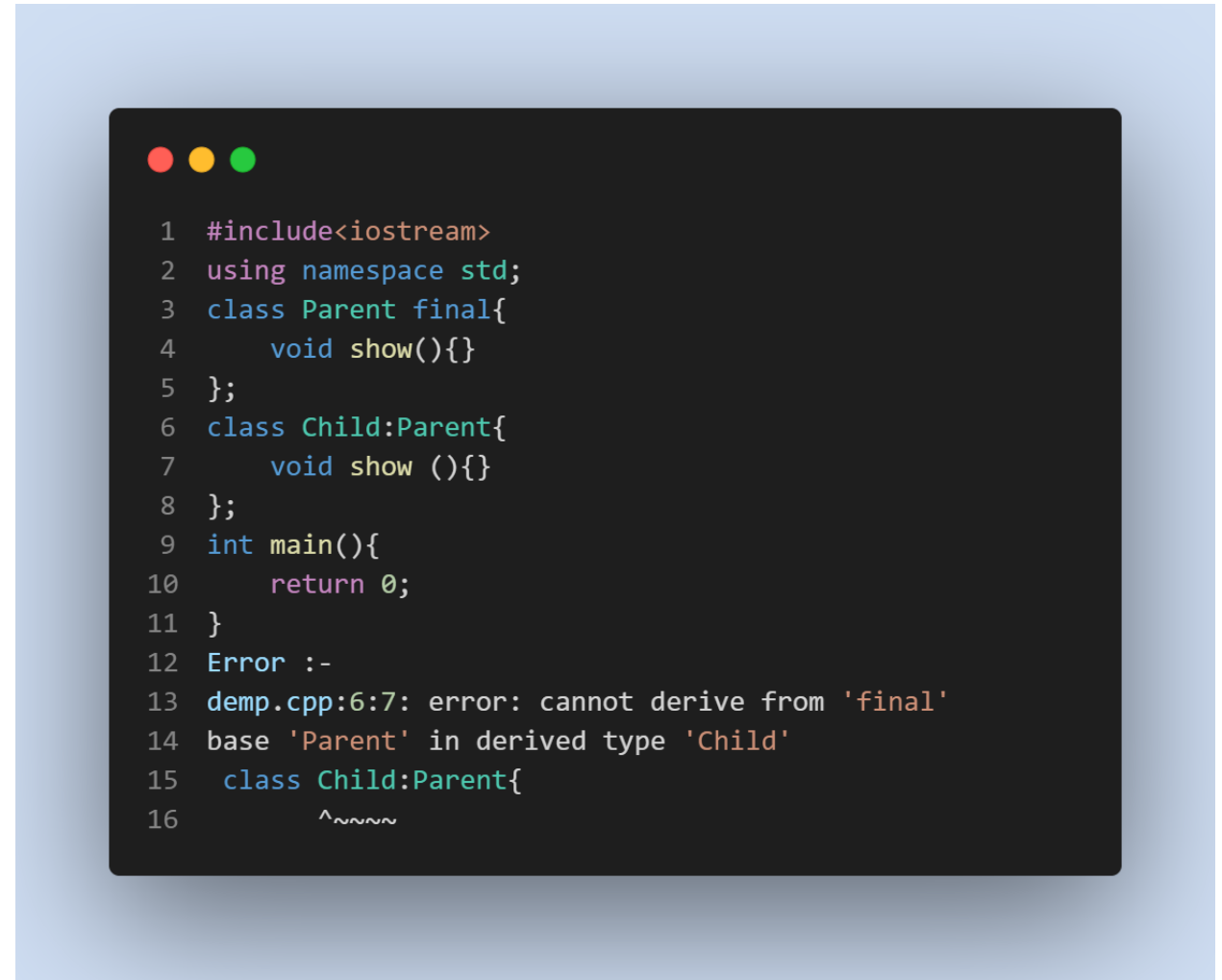
```
1  #include<iostream>
2  using namespace std;
3  int main(){
4      float x = 32.2;
5      decltype(x) z = 67.8;
6      cout << x << endl;
7      cout << z << endl;
8      return 0;
9  }
10 //Output
11 32.2
12 67.8
```

Final Keyword in C++

Mainly the final keyword is used for two purposes. They are as follows :-

- Restrict Class Inheritance
- Restrict Method Overriding

Restrict Class Inheritance :-



```
1  #include<iostream>
2  using namespace std;
3  class Parent final{
4      void show(){}
5  };
6  class Child:Parent{
7      void show (){}
8  };
9  int main(){
10     return 0;
11 }
12 Error :-
13 demp.cpp:6:7: error: cannot derive from 'final'
14 base 'Parent' in derived type 'Child'
15 class Child:Parent{
16     ^~~~~~
```

Restrict Method Overriding :-

```
1  #include<iostream>
2  using namespace std;
3  class Parent{
4      virtual void show() final{}
5  };
6  class Child : Parent{
7      void show(){}
8  };
9  int main(){
10     return 0;
11 }
12 demp.cpp:4:18: error: overriding
13 final function 'virtual void Parent::show()'
14     virtual void show() final{}
15                     ^~~~~
```

Lambda Expressions :- Lambda Expressions is useful for defining unnamed functions in C++. So, we can define the function wherever we like. They are more likely inline functions but they will have the features of the function. Let us see the syntax for a lambda expression or unnamed function in C++.

[Capture_List] (Parameter_List) -> Return_Type { body };

```
#include <iostream>
using namespace std;
int main(){
    [](){cout << "Hello"<<endl;}();
    [](int x, int y) {cout << "Sum: " << x + y <<endl;}(10, 5);
    int x = [](int x, int y) {return x+y;} (10, 5);
    cout<<x;
}
```

Output :-

```
Hello
Sum: 15
15
```

So, for calling this function we can directly put the parenthesis '()' after the curly braces.

Another way of Calling Unnamed Function in C++ :-

```
#include <iostream>
using namespace std;
int main(){
    auto F = [](){cout << "Hello";}
    F();
}
```

Writing Return Type using Lambda Expression in C++:

By default, the C++ compiler will identify the return type, so we don't have to mention it or if you want then you can mention it.

```
#include <iostream>
using namespace std;
int main(){
    int S = [] (int x, int y) -> int{return x + y;}(10, 5);
    cout<<S;
}
```

Output: 15

Accessing local variables of a function inside the unnamed function:

```
#include <iostream>
using namespace std;
int main(){
    int x = 3;
    int y = 6;
    [x, y] () {cout << x << " " << y;}();
}
```

`[x, y] () {cout << ++x << " " << ++y;};` Can we write like this? No. We cannot modify these captured variables. For modifying, we have to use a reference for capturing variables as follows:

`[&x, &y] () {cout << ++x << " " << ++y;};` Now we can modify the x and y values. By writing reference, we can modify the local variables inside the lambda expression. The complete example code is given below.

If we want to access all the things in this scope then just write a reference inside the capture list as follows:

```
[&] () {cout << ++x << " " << ++y;};
```

```
1  #include<iostream>
2  using namespace std;
3  template<typename t>
4  void fun(t r){
5      r();
6  }
7  int main(){
8      cout << "Function 1:" << endl;
9      [](){cout << "Hello";}();
10
11     cout << endl << "Function 2:" << endl;
12     int z = [](int x, int y) {return x+y;} (30, 32);
13     cout << "Sum: " << z;
14
15     cout << endl << "Function 3:" << endl;
16     int x = 5;
17     int y = 6;
18
19     auto F = [&x, &y](){cout << ++x << " " << ++y << endl;};
20     cout << "x & y : ";
21     F();
22
23     cout << endl << "Function 4:" << endl;
24     auto S = [x, y](){cout << "x * y: " << x*y << endl;};
25     fun(S);
26     return 0;
27 }
28
```

Output :-

```
Function 1:
Hello
Function 2:
Sum: 62
Function 3:
x & y : 6 7

Function 4:
x * y: 42
```

Disadvantages of Pointers :-

- Uninitialized Pointers
- The pointer may cause a memory leak
- Dangling Pointers

Uninitialized pointers in C++ :-

```
int *p;
```

if we have declared a pointer then we should not use that pointer unless we have initialized it.

```
*p = 25;
```

This means that we want to store the value '25' wherever the pointer is pointing. But the question here is where the pointer is pointing? In, 'int *p' it is not pointing anywhere. Just we have declared. Then what is the address in 'p' here?

Some default garbage addresses may be in 'p', some random addresses that may belong to a program or may not belong to a problem. So, it is an invalid address as we have not made a pointer to point to some particular location. So first of all, make it point to some location then we can access it. There are 3 methods of doing it.

1st Method :-

```
int x = 10;          int *p = &x;
```

If we have some variable 'x' then, Now the pointer is pointing to this known variable 'x' which is already declared.

2nd Method :-

```
int *p = (int*) 0x5628;
```

We can assign some addresses using some hexadecimal codes but that address has to be type casted as an integer pointer. So, can we directly assign some addresses to a pointer? Yes, if we are sure that the address belongs to the program so this type of initialization is also allowed. **This is not commonly used. This is mostly used in systems programming**

3rd Method :-

```
int *p = new int[5];
```

We can dynamically allocate some memory and assign that to a pointer. If we don't write size and write only 'int' then it will allocate just one integer so either to an existing variable.

Memory Leak :-

This is related to a pointer as well as heap memory. As we have already discussed heap memory, when we are allocating heap memory then when we don't require that, we should deallocate it. If we don't deallocate it then we say that memory is leaked from that total set of memory.

```
int *p = new int[3];           p = NULL;
```

Here we have a pointer and I have allocated heap memory of some size. Then after some time, we don't need this memory. So, we simply say 'p = NULL', then point P will not be pointing on that memory. We should not do this unless we have explicitly deleted the memory. So first of all, say delete 'p' then only, make 'p' as null.

```
delete []p;
```

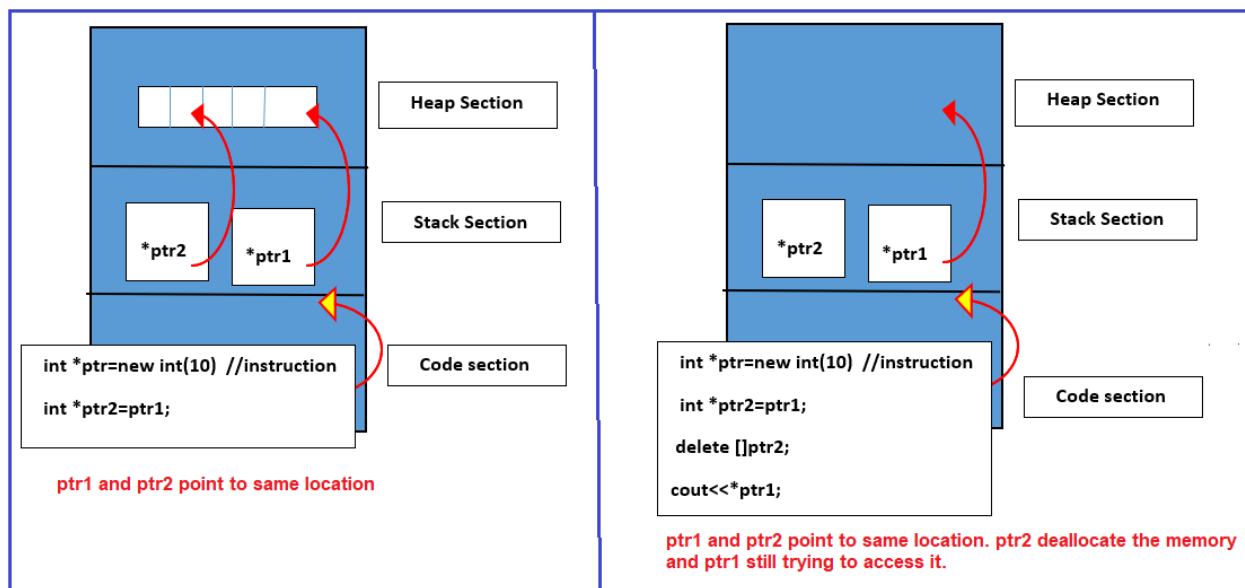
```
p = NULL;
```

Now here is one more thing that we can write 'p = 0' also or write 'p = nullptr'.

In modern C++ it is suggested to use 'nullptr'. You should avoid using null. So back to this memory leak, the conclusion is you must delete the memory when you are not using it before making a pointer to null. Now let us move to the third problem which is the dangling pointer.

Dangling Pointer in C++ :-

So uninitialized pointers mean, the pointer is never initialized, dangling pointer means the pointer was initialized but memory is deallocated. We should avoid these three types of problems while writing programs or developing applications. Actually, these problems are caused due to negligence of beginner programmers. Expert programmers may check all these things thoroughly only before delivering a program or before developing software.



```
#include<iostream>
using namespace std;
int main() {
    int *ptr1=new int(10);
    int *ptr2=ptr1;
    delete []ptr2;
    cout<<*ptr1;
    return 0;
}
```

The improper use of pointers is often at the root of many security problems. When an application behaves in unpredictable ways, it may not seem to be a security issue, at least in terms of unauthorized access.

Pointers can do arithmetic, References can't: **Memory access via pointer arithmetic is fundamentally unsafe and for safeguarding,** Java has a robust security model and disallows pointer arithmetic for this reason.

What are Smart Pointers in C++?

If you are creating anything inside the heap memory then for accessing the heap memory, we need to use pointers. The problem with heap memory is that when you don't need it then you must deallocate the memory. And mostly the programmer shows laziness in writing the code for the deallocation of objects from heap memory which causes severe problems like memory leaks which will cause the program to crash.



```
1  #include<iostream>
2  using namespace std;
3  class Rectangle{
4  private:
5      int Length;
6      int Breadth;
7  public:
8      Rectangle(int l, int b){
9          Length = l;
10         Breadth = b;
11     }
12     int Area(){
13         return Length * Breadth;
14     }
15 };
16
17 int Fun(int l, int b){
18     Rectangle *p = new Rectangle(l, b);
19     int area = p->Area();
20     delete p;
21     return area;
22 }
23 int main(){
24     while(1){
25         int Result = Fun(10, 20);
26         cout << Result << endl;
27     }
28     return 0;
29 }
```

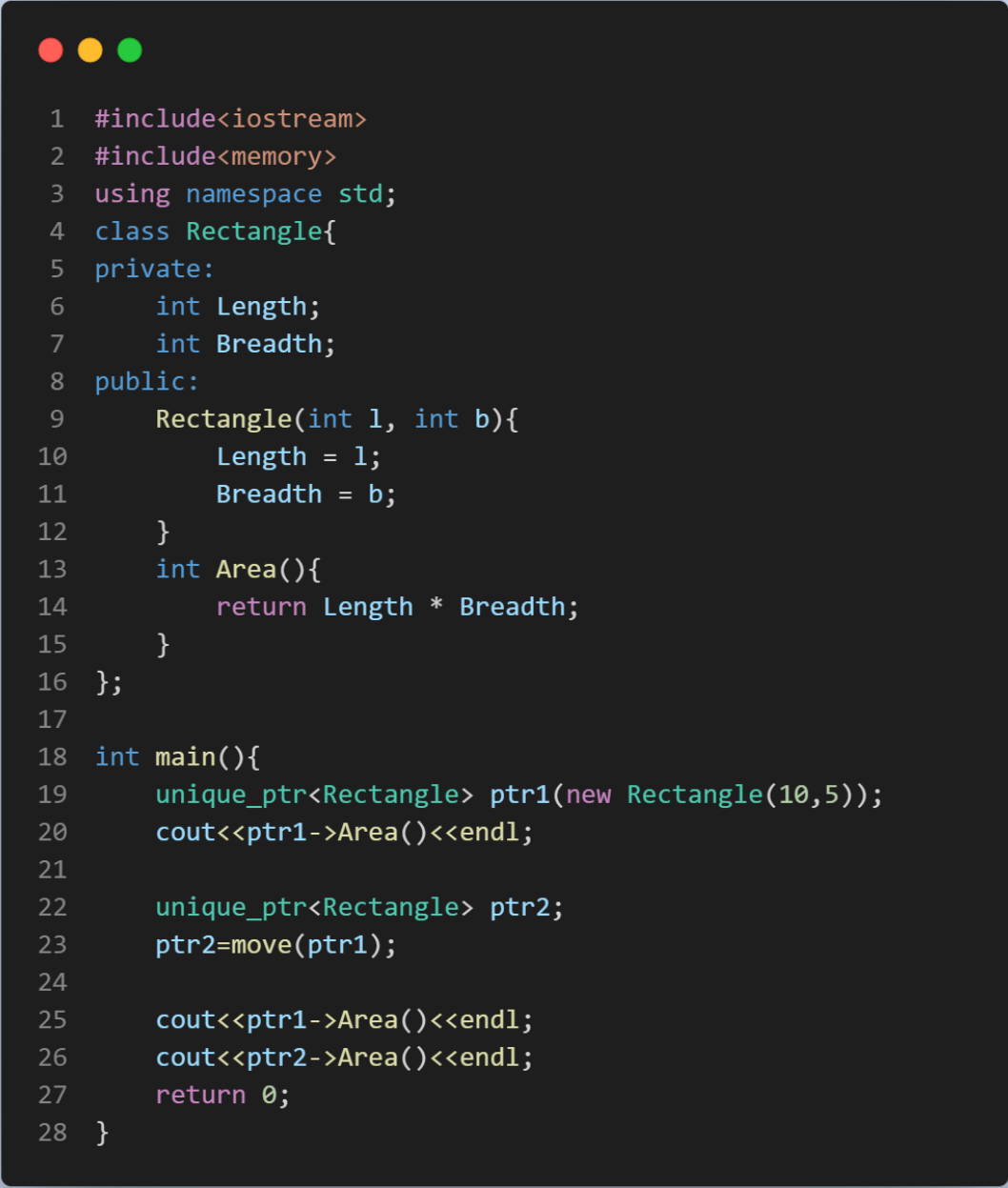
Using Smart Pointers in C++:

If we declare the smart pointer then they will automatically deallocate the object when the smart pointer is going out of the scope. Let us show you how we can declare smart pointers in C++.

```
int Fun(int l, int b)
{
    unique_ptr<Rectangle> p(new Rectangle(l, b));
    int area = p->Area();
    return area;
}
```

Example to Understand unique_ptr in C++ :-

If you are using unique_ptr, if an object is created and a pointer is pointing to that object then only one pointer can point to that object. So, we cannot share this object with another pointer. But we can transfer the control from one pointer to another pointer by removing p1. So unique_ptr means upon an object at a time only one pointer will be pointing.



```
1  #include<iostream>
2  #include<memory>
3  using namespace std;
4  class Rectangle{
5  private:
6      int Length;
7      int Breadth;
8  public:
9      Rectangle(int l, int b){
10         Length = l;
11         Breadth = b;
12     }
13     int Area(){
14         return Length * Breadth;
15     }
16 };
17
18 int main(){
19     unique_ptr<Rectangle> ptr1(new Rectangle(10,5));
20     cout<<ptr1->Area()<<endl;
21
22     unique_ptr<Rectangle> ptr2;
23     ptr2=move(ptr1);
24
25     cout<<ptr1->Area()<<endl;
26     cout<<ptr2->Area()<<endl;
27     return 0;
28 }
```

Output: 50

Shared_ptr :-

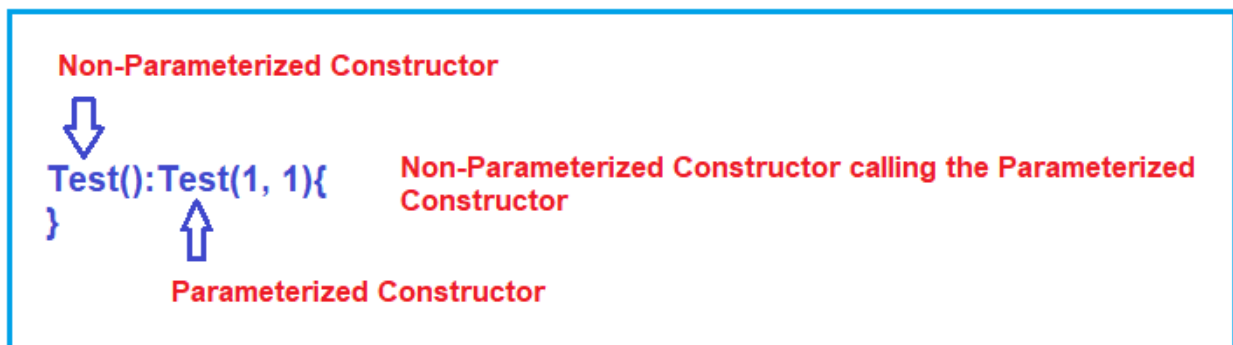
Just like how we have used `unique_ptr`, the same way we have to use `shared_ptr`. More than one pointer can point to one object. This pointer maintains a `Ref_count` that is a reference counter. Suppose 3 pointers are pointing on a single object the `Ref_count` will be 3. So `shared` means an object can be used by more than one pointer. If we remove one pointer then `Ref_count` will be reduced by 1. We can know the value of `Ref_count` by using the `use_count()` function.

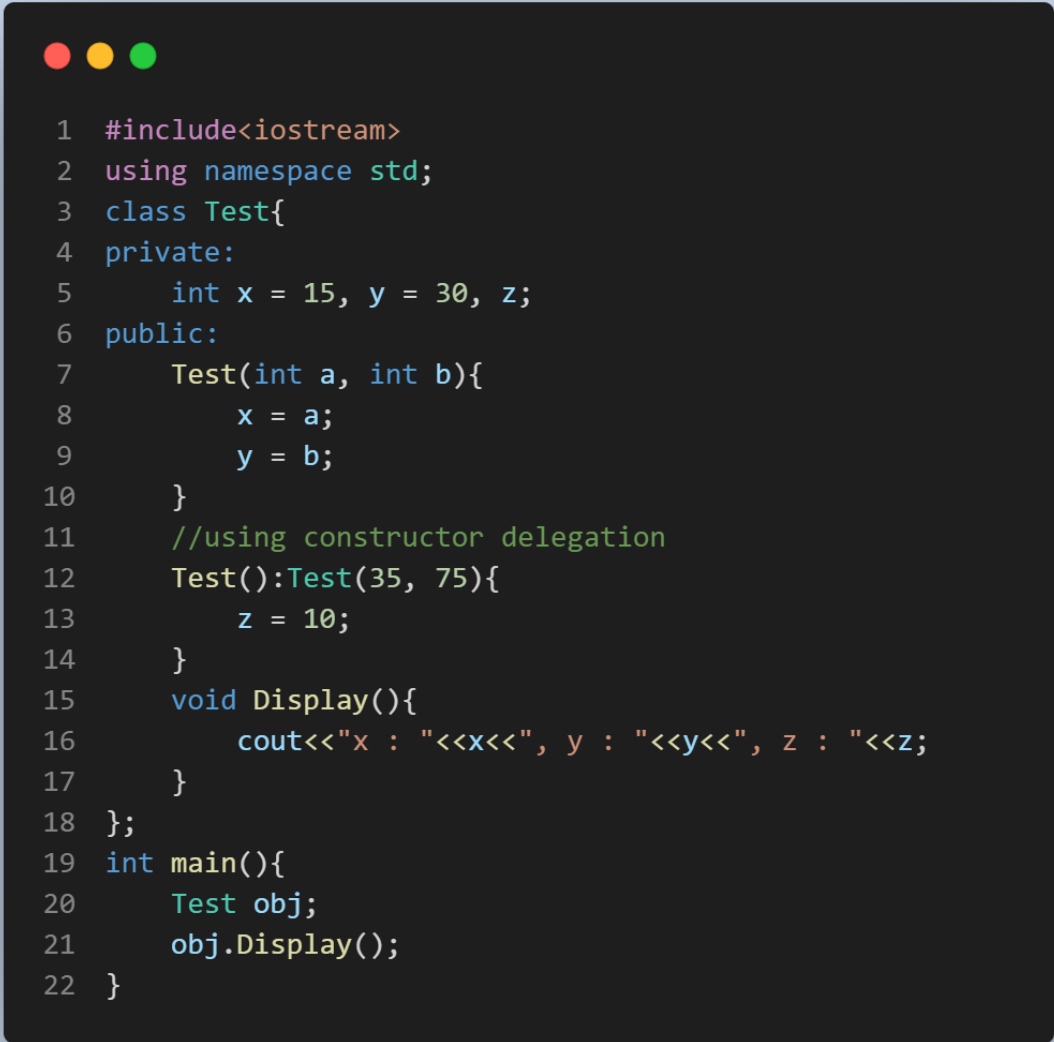
```
1  #include<iostream>
2  #include<memory>
3  using namespace std;
4  class Rectangle{
5  private:
6      int Length;
7      int Breadth;
8  public:
9      Rectangle(int l, int b){
10         Length = l;
11         Breadth = b;
12     }
13     int Area(){
14         return Length * Breadth;
15     }
16 };
17
18 int main(){
19     shared_ptr <Rectangle> ptr1 (new Rectangle(10, 5));
20     cout << ptr1->Area() << endl;
21
22     shared_ptr <Rectangle> ptr2;
23     ptr2 = ptr1;
24
25     cout << "ptr1 " << ptr1->Area() << endl;
26     cout << "ptr1 " << ptr2->Area() << endl;
27     cout << ptr1.use_count() << endl;
28 }
29 //Output
30 50
31 ptr1 50
32 ptr1 50
33 2
```

Weak_ptr :-

It is also the same as shared_ptr. Here also more than one pointer can point to a single object. But it will not maintain Ref_count. So that's why it is known as weak_ptr. So, the pointer will not have a strong hold on the object. The reason is if the pointers are holding the object and requesting other objects, they may form a deadlock between the pointers. So, to avoid deadlock, weak_ptr is useful. So, it doesn't have Ref_count so it is more like unique_ptr but it allows the pointer to share an object, so it is more like shared_ptr. It is in between unique and shared which is not strict. It doesn't bother how many pointers are pointing at an object.

InClass Initializer and Delegation of Constructors :-





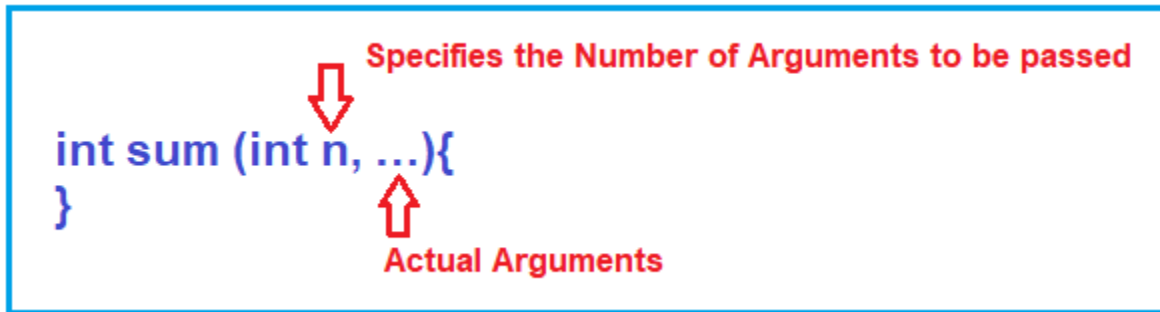
```
1  #include<iostream>
2  using namespace std;
3  class Test{
4  private:
5      int x = 15, y = 30, z;
6  public:
7      Test(int a, int b){
8          x = a;
9          y = b;
10     }
11     //using constructor delegation
12     Test():Test(35, 75){
13         z = 10;
14     }
15     void Display(){
16         cout<<"x : "<<x<<", y : "<<y<<", z : "<<z;
17     }
18 };
19 int main(){
20     Test obj;
21     obj.Display();
22 }
```

Ellipsis in C++:

Ellipsis is used for taking a variable number of arguments in a function. For example, if we want to write a function for finding the sum of elements or sum of integers then we want our function to work with a different number of parameters like,

Sum (10, 20) = 30

Sum (30, 54, 23) = 107



How to access Elements using Ellipsis in C++?

For accessing the elements, there is a class available in C++ that is `va_list`. For a better understanding, please have a look at the following code. What are the important instructions in this function?

`va_list()`, `va_start()`, `va_arg()` and `va_end()`.

```
int Sum(int n, ...){  
    va_list list;  
    va_start(list, n);  
    int s = 0;  
  
    for(int i = 0; i < n; i++)  
        s += va_arg(list, int);  
  
    va_end(list);  
    return s;  
}
```



```
1  #include<iostream>
2  #include<cstdarg>
3  using namespace std;
4  int sum (int n, ...){
5      va_list list;
6      va_start (list, n);
7      int x;
8      int s = 0;
9      for (int i = 0; i < n; i++){
10         x = va_arg (list, int);
11         s += x;
12     }
13     return s;
14 }
15 int main(){
16     cout << sum (3, 12, 24, 36) << endl;
17     cout << sum (7, 13, 26, 39, 52, 65, 78, 81) << endl;
18 }
19 //Output
20 72
21 354
```